

BLOCKCHAIN AUDIT REPORT

Pharos BlockChain

DECEMBER 2025

Contents

1. EXECUTIVE SUMMARY	6
1.1 Methodology	6
2. FINDINGS OVERVIEW	10
2.1 Project Info And Contract Address	10
2.2 Summary	10
2.3 Key Findings	11
3. DETAILED DESCRIPTION OF FINDINGS	18
3.1 Critical-Integer Underflow in Gas Calculation	18
3.2 Critical-Unhandled Exceptions in Modexp::Execute, It will lead to the node DOS	19
3.3 Critical- DoS in Transaction::GetType()	20
3.4 Critical- Use of Unvalidated Data After Failed Deserialization in FromSSZ	21
3.5 Critical-Improper Error Handling in Signature Recovery	22
3.6 Critical-Insecure Key Generation in SetKey	24
3.7 Critical-Empty Keypair Accepted, Leading to Uninitialized Signer Object	25
3.8 Critical-Denial of Service in Utils::MultiVerify	26
3.9 Critical-Race Condition Leading to Use-After-Free in SelServer	29
3.10 Critical-Buffer Bounds Check Removed in release build	31
3.11 Critical-Unsafe Deserialization in ReceiverFromRlp	32
3.12 Critical-DOG Service Data Race in Consensus Node List Access	33
3.13 Critical-TOCTOU Race Condition in DDosDefender::ForwardInc	35
3.14 Critical-Inconsistent Lock Usage in FetchingLock	36
3.15 Critical-Race Condition in Rule State Updates	38
3.16 Critical- Race Condition in Bloom Filter Update	39
3.17 Critical-Unauthorized Access to Debug APIs Allows Remote Configuration Modification	41
3.18 High-Private Key Exposure in Logs	44
3.19 High-State Inconsistency in Withdrawal Mechanism Leads to Permanent Fund Loss	45
3.20 High-Invalid Address validation	47
3.21 High-Missing Error Handling, may lead to the node crash	48
3.22 High-State Inconsistency in Withdrawal Mechanism Leads to Permanent Fund Loss	49
3.23 High-Hardcoded Password in Cryptographic Key Operations	54
3.24 High-DOG Service Timer Cancellation Failure Leading to Post-Shutdown Execution	56
3.25 High-Cache Miss Handling Inadequacy Causes State Desynchronization	59
3.26 High-uninitialized objects in function AggregateKeypair	62
3.27 High-AggregableKeypair's pubkey guard inverted	63

3.28 High- Private key bytes not securely erased	64
3.29 High-missing BLS serialization failure handling	66
3.30 High-AggregableKeypair objects aggregated	68
3.31 High-Recursive Stack Overflow in CallTraceToJson	70
3.32 High-Sensitive Key Material Not Cleared from Memory	71
3.33 Medium-Race Condition in Task Counting	72
3.34 Medium-Unsafe Busy Wait in Destructor	73
3.35 Medium-Iterator Dereference After End Check	74
3.36 Medium-Infinite Wait in ReadBlockSeqMap During Initialization	76
3.37 Medium-Missing Capacity Check in ABuffer CutTail Method	79
3.38 Medium-DOG Service Array Bounds Checking Leading to DoS	80
3.39 Medium-DOG Service DNS Hijacking Leading to DoS/Eclipse Attack	82
3.40 Medium-DOG Service Unauthenticated Node Addition Leading to Resource Exhaustion	84
3.41 Medium-DOG Service Unbounded Pending Connections Leading to Resource Exhaustion	87
3.42 Medium-Rate Limiting Logic Flaw in ETHSendRawTransaction Allows DoS Bypass	90
3.43 Medium-Rate Limiter Initialized But Never Used	91
3.44 Medium-ETH Call Pending State Implementation Violates Ethereum JSON-RPC Specification	92
3.45 Medium-Unsafe File Permissions Due to umask(0) in Daemon Mode	94
3.46 Medium-Component Shutdown Failures Ignored in ShutdownComponents	96
3.47 Medium-CreateSessionForHttpServer Bypasses Session Limit	98
3.48 Medium-Null Pointer Dereference in EVMTxGenerator::SetEvmType	100
3.49 Medium-Hardcoded Timezone Offset in Time Utility Functions	102
3.50 Medium-Buffer Overflow in BitMapper::GenerateBitmap	104
3.51 Medium-AES-GCM IV Length Parameter Inconsistency	106
3.52 Medium-AES-GCM Random Key Generation Only Assigns First Byte	108
3.53 Medium-RSA-OAEP Encryption and Decryption Use Incompatible Encoding	109
3.54 Low- Unused Buffer with Potential Memory Disclosure	110
3.55 Low-Super Permission Design Flaw in InferRuleManager	111
3.56 Low-keys repeated coverage	113
3.57 Low-Out-of-Bounds Access Vulnerability in OnSysStakingEvent	115
3.58 Low-SSZ Data Integrity Validation Bypass	117
3.59 Low-Invalid BLS Public Key Processing Continuation	119
3.60 Low-Configuration Validation Error Handling Inconsistency	121
3.61 Low-DOG Service Resource Cleanup Failure on Initialization Error	123
3.62 Low-DOG Service Division by Zero in Consensus Node List Access	125
3.63 Low-Race Condition in BroadcastMessage Leading to Resource Limit Bypass	127
3.64 Low-Session Rate Limiting Config Parameter Mapping Error	129

3.65 Low-HealthChecker Indefinite Wait Causes DoS on Unresponsive RPC	131
3.66 Low-ETH Subscribe Logs DoS	133
3.67 Low-Internal RPC Call Missing Timeout Handling	134
3.68 Low-SSZViews Function Out-of-Bounds Read	136
3.69 Low-ComboChannel::AddServer Null Pointer Dereference	138
3.70 Low-Undefined Behavior in Metadata Processing	139
3.71 Low-RecvBlockProof Unauthenticated Block Proof Acceptance	141
3.72 Low-Spec Version Misuse in SSZ Decoding	143
3.73 Low-Incomplete State Synchronization in TxpoolService	145
3.74 Low-Incorrect Metrics Counting in EnqueueProposeTx	146
3.75 Low-Inconsistent Boundary Condition in HandleMetricsCollect	148
3.76 Low-Incorrect Shutdown Status Semantics in IsShutdown()	149
3.77 Low-Unimplemented Partition Management Functions	151
3.78 Low-Infinite Loop in Process Termination	152
3.79 Low-Silent Failure When ThreadPool Size is Zero	154
3.80 Low-Use-After-Free Risk in AddRepeating with Auto Delete Callbacks	156
3.81 Low-Null Pointer Dereference in ChainUtils::GetBasicGas	158
3.82 Low-Unchecked Vector Access in EVM State Functions	160
3.83 Low-Inconsistent Return Values in GenerateFromTemplate Methods	162
3.84 Low-Inadequate Error Handling in Rdtsc Function	164
3.85 Low-Missing Exception Handling in JsonToFormatString	166
3.86 Low-Buffer Overflow in Readu_be	168
3.87 Low-Incorrect Null Pointer Check in SM2 SetPubkey	171
3.88 Info-Gas-Validator Struct Fields Are Not Packed	173
3.89 Info-Gas-Optimizing nested loop code	175
3.90 Info-Gas Opt-Inefficient Array Traversal in advanceEpoch Function	177
3.91 Info-The Function MatchTopic argument topic is useless in this function. It will always return true	178
3.92 Info-Useless function	179
3.93 Info-Unused Parameter in function MerkleTree::Root()	180
3.94 Info-Replace hard-coded value	181
3.95 Info-Incomplete Log Information in StartupImpl	182
3.96 Info-NeedBypassFork Unnecessary Call in Production Environment	184
3.97 Info-std::cout Error Output Stream Inconsistency	186
3.98 Info-Inconsistent Module Name in Error Log	187
3.99 Info-Incorrect Service Name in Process Lock Error Message	190
3.100 Info-Invalid Conditional Statement in EVMTxGenerator::GenerateTransaction	191
3.101 Info-Incorrect Error Codes in Ciphertext Parsing Functions	192

4. CONCLUSION	194
5. APPENDIX	195
5.1 Basic Coding Assessment	195
5.1.1 Apply Verification Control	195
5.1.2 Authorization Access Control	195
5.1.3 Forged Transfer Vulnerability	195
5.1.4 Transaction Rollback Attack	196
5.1.5 Transaction Block Stuffing Attack	196
5.1.6 Soft Fail Attack Assessment	196
5.1.7 Hard Fail Attack Assessment	197
5.1.8 Abnormal Memo Assessment	197
5.1.9 Abnormal Resource Consumption	197
5.1.10 Random Number Security	198
5.2 Advanced Code Scrutiny	198
5.2.1 Cryptography Security	198
5.2.2 Account Permission Control	198
5.2.3 Malicious Code Behavior	199
5.2.4 Sensitive Information Disclosure	199
5.2.5 System API	199
6. DISCLAIMER	200
7. REFERENCES	201

1. EXECUTIVE SUMMARY

ExVul Web3 Security was engaged by **Pharos** to review blockchain implementation. The assessment was conducted in accordance with our systematic approach to evaluate potential security issues based upon customer requirement. The report provides detailed recommendations to resolve the issue and provide additional suggestions or recommendations for improvement.

The outcome of the assessment outlined in chapter 3 provides the system's owners a full description of the vulnerabilities identified, the associated risk rating for each vulnerability, and detailed recommendations that will resolve the underlying technical issue.

1.1 Methodology

To standardize the evaluation, we define the following terminology based on OWASP Risk Rating Methodology [10] which is the gold standard in risk assessment using the following risk models:

- **Likelihood:** represents how likely a particular vulnerability is to be uncovered and exploited in the wild.
- **Impact:** measures the technical loss and business damage of a successful attack.
- **Severity:** determine the overall criticality of the risk.

Likelihood can be: High, Medium and Low and impact are categorized into: High, Medium, Low, Informational. Severity is determined by likelihood and impact and can be classified into five categories accordingly: Critical, High, Medium, Low, Informational shown in table 1.1.

		Informational	Low	Medium	High
Likelihood	High	INFO	MEDIUM	HIGH	CRITICAL
	Medium	INFO	LOW	MEDIUM	HIGH
	Low	INFO	LOW	LOW	MEDIUM
		IMPACT			

Table 1.1 Overall Risk Severity

To evaluate the risk, we will be going through a list of items, and each would be labelled with a severity category. The audit was performed with a systematic approach guided by a comprehensive assessment list carefully designed to identify known and impactful security issues. If our tool or analysis does not identify any issue, the contract can be considered safe regarding the assessed item. For any discovered issue, we might further deploy contracts on our private test environment and run tests to confirm the findings. If necessary, we would additionally build a PoC to demonstrate the possibility of exploitation. The concrete list of check items is shown in Table 1.2.

- **Basic Coding Bugs:** We first statically analyze given Blockchain with our proprietary static code analyzer for known coding bugs, and then manually verify (reject or confirm) all the issues found by our tool.
- **Code and business security testing:** We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.
- **Additional Recommendations:** We also provide additional suggestions regarding the coding and development of Blockchains from the perspective of proven programming practices.

Category	Assessment Item
P2P Communication Security	• Connection Number Occupation Audit • Eclipse Attack • Packet Size Limit • Node Communication Protocol Security
RPC Interface Security	• RPC Sensitive Interface Permissions • Traditional Web Security • RPC Interface Security

Consensus Mechanism Security	• Design Of Consensus Mechanism • Implementation Of Consensus Verification • Incentive Mechanism Audit
Transaction processing Security	• Transaction Signature Logic • Transaction Verification Logic • Transaction Processing Logic • Transaction Fee Setting • Transaction Replay
Cryptography Security	• Random Number Range And Probability Distribution • Cryptographic Algorithm Implementation/Use
Wallet Module & Account Security Audit	• Private Key / Mnemonic Word Storage Security • Private Key / Mnemonic Word Usage Security • Private key/mnemonic generation algorithm

Others Security Audit	• Database Security • Thread Security • File Permission Security • Historical Vulnerability Security
------------------------------	--

Table 1.2: The Full List of Assessment Items

To better describe each issue we identified, we categorize the findings with Common Weakness Enumeration (CWE-699) [14], which is a community-developed list of software weakness types to better delineate and organize weaknesses around concepts frequently encountered in software development.

2. FINDINGS OVERVIEW

2.1 Project Info And Contract Address

Project Name	Audit Time	Language
Pharos	20/07/2025 - 01/12/2025	C++/Go/Solidity

Repository

https://github.com/PharosNetwork/AntChainAldabaNG/tree/cyrus/mainnet_audit

Commit Hash

1d3da4658ec9efd0e519a9291c41ee8ab4a56264

2.2 Summary

Severity	Found
CRITICAL	17
HIGH	15
MEDIUM	21
LOW	34
INFO	14



2.3 Key Findings

Severity	Findings Title	Status
CRITICAL	Critical-Integer Underflow in Gas Calculation	Fixed
CRITICAL	Critical-Unhandled Exceptions in Modexp::Execute, It will lead to the node DOS	Fixed
CRITICAL	Critical- DoS in Transaction::GetType()	Fixed
CRITICAL	Critical- Use of Unvalidated Data After Failed Deserialization in FromSSZ	Fixed
CRITICAL	Critical-Improper Error Handling in Signature Recovery	Fixed
CRITICAL	Critical-Insecure Key Generation in SetKey	Fixed
CRITICAL	Critical-Empty Keypair Accepted, Leading to Uninitialized Signer Object	Fixed
CRITICAL	Critical-Denial of Service in Utils::MultiVerify	Fixed
CRITICAL	Critical-Race Condition Leading to Use-After-Free in SelServer	Fixed
CRITICAL	Critical-Buffer Bounds Check Removed in release build	Fixed
CRITICAL	Critical-Unsafe Deserialization in ReceiverFromRlp	Fixed
CRITICAL	Critical-DOG Service Data Race in Consensus Node List Access	Fixed
CRITICAL	Critical-TOCTOU Race Condition in DDosDefender::ForwardInc	Fixed
CRITICAL	Critical-Inconsistent Lock Usage in FetchingLock	Fixed
CRITICAL	Critical-Race Condition in Rule State Updates	Fixed
CRITICAL	Critical- Race Condition in Bloom Filter Update	Fixed
CRITICAL	Critical-Unauthorized Access to Debug APIs Allows Remote Configuration Modification	Fixed

Severity	Findings Title	Status
HIGH	High-Private Key Exposure in Logs	Fixed
HIGH	High-State Inconsistency in Withdrawal Mechanism Leads to Permanent Fund Loss	Fixed
HIGH	High-Invalid Address validation	Fixed
HIGH	High-Missing Error Handling, may lead to the node crash	Fixed
HIGH	High-State Inconsistency in Withdrawal Mechanism Leads to Permanent Fund Loss	Fixed
HIGH	High-Hardcoded Password in Cryptographic Key Operations	Fixed
HIGH	High-DOG Service Timer Cancellation Failure Leading to Post-Shutdown Execution	Fixed
HIGH	High-Cache Miss Handling Inadequacy Causes State Desynchronization	Fixed
HIGH	High-uninitialized objects in function AggregateKeypair	Fixed
HIGH	High-AggregableKeypair's pubkey guard inverted	Fixed
HIGH	High- Private key bytes not securely erased	Fixed
HIGH	High-missing BLS serialization failure handling	Fixed
HIGH	High-AggregableKeypair objects aggregated	Fixed
HIGH	High-Recursive Stack Overflow in CallTraceToJson	Fixed
HIGH	High-Sensitive Key Material Not Cleared from Memory	Fixed
MEDIUM	Medium- Race Condition in Task Counting	Fixed
MEDIUM	Medium-Unsafe Busy Wait in Destructor	Fixed
MEDIUM	Medium-Iterator Dereference After End Check	Fixed

Severity	Findings Title	Status
MEDIUM	Medium-Infinite Wait in ReadBlockSeqMap During Initialization	Fixed
MEDIUM	Medium-Missing Capacity Check in ABuffer CutTail Method	Fixed
MEDIUM	Medium-DOG Service Array Bounds Checking Leading to DoS	Fixed
MEDIUM	Medium-DOG Service DNS Hijacking Leading to DoS/Eclipse Attack	Fixed
MEDIUM	Medium-DOG Service Unauthenticated Node Addition Leading to Resource Exhaustion	Fixed
MEDIUM	Medium-DOG Service Unbounded Pending Connections Leading to Resource Exhaustion	Fixed
MEDIUM	Medium-Rate Limiting Logic Flaw in ETHSendRawTransaction Allows DoS Bypass	Fixed
MEDIUM	Medium-Rate Limiter Initialized But Never Used	Fixed
MEDIUM	Medium-ETH Call Pending State Implementation Violates Ethereum JSON-RPC Specification	Fixed
MEDIUM	Medium-Unsafe File Permissions Due to umask(0) in Daemon Mode	Fixed
MEDIUM	Medium-Component Shutdown Failures Ignored in ShutdownComponents	Fixed
MEDIUM	Medium-CreateSessionForHttpServer Bypasses Session Limit	Fixed
MEDIUM	Medium-Null Pointer Dereference in EVMTxGenerator::SetEvmType	Fixed
MEDIUM	Medium-Hardcoded Timezone Offset in Time Utility Functions	Fixed
MEDIUM	Medium-Buffer Overflow in BitMapper::GenerateBitmap	Fixed
MEDIUM	Medium-AES-GCM IV Length Parameter Inconsistency	Fixed
MEDIUM	Medium-AES-GCM Random Key Generation Only Assigns First Byte	Fixed

Severity	Findings Title	Status
MEDIUM	Medium-RSA-OAEP Encryption and Decryption Use Incompatible Encoding	Fixed
LOW	Low-Unused Buffer with Potential Memory Disclosure	Fixed
LOW	Low-Super Permission Design Flaw in InferRuleManager	Fixed
LOW	Low-keys repeated coverage	Fixed
LOW	Low-Out-of-Bounds Access Vulnerability in OnSysStakingEvent	Fixed
LOW	Low-SSZ Data Integrity Validation Bypass	Fixed
LOW	Low-Invalid BLS Public Key Processing Continuation	Fixed
LOW	Low-Configuration Validation Error Handling Inconsistency	Fixed
LOW	Low-DOG Service Resource Cleanup Failure on Initialization Error	Fixed
LOW	Low-DOG Service Division by Zero in Consensus Node List Access	Fixed
LOW	Low-Race Condition in BroadcastMessage Leading to Resource Limit Bypass	Fixed
LOW	Low-Session Rate Limiting Config Parameter Mapping Error	Fixed
LOW	Low-HealthChecker Indefinite Wait Causes DoS on Unresponsive RPC	Acknowledge
LOW	Low-ETH Subscribe Logs DoS	Acknowledge
LOW	Low-Internal RPC Call Missing Timeout Handling	Acknowledge
LOW	Low-SSZViews Function Out-of-Bounds Read	Fixed
LOW	Low-ComboChannel::AddServer Null Pointer Dereference	Fixed
LOW	Low-Undefined Behavior in Metadata Processing	Fixed

Severity	Findings Title	Status
LOW	Low-RecvBlockProof Unauthenticated Block Proof Acceptance	Acknowledge
LOW	Low-Spec Version Misuse in SSZ Decoding	Acknowledge
LOW	Low-Incomplete State Synchronization in TxpoolService	Acknowledge
LOW	Low-Incorrect Metrics Counting in EnqueueProposeTxs	Fixed
LOW	Low-Inconsistent Boundary Condition in HandleMetricsCollect	Fixed
LOW	Low-Incorrect Shutdown Status Semantics in IsShutdown()	Fixed
LOW	Low-Unimplemented Partition Management Functions	Fixed
LOW	Low-Infinite Loop in Process Termination	Acknowledge
LOW	Low-Silent Failure When ThreadPool Size is Zero	Fixed
LOW	Low-Use-After-Free Risk in AddRepeating with Auto Delete Callbacks	Fixed
LOW	Low-Null Pointer Dereference in ChainUtils::GetBasicGas	Fixed
LOW	Low-Unchecked Vector Access in EVM State Functions	Fixed
LOW	Low-Inconsistent Return Values in GenerateFromTemplate Methods	Fixed
LOW	Low-Inadequate Error Handling in Rdtsc Function	Fixed
LOW	Low-Missing Exception Handling in JsonToFormatString	Fixed
LOW	Low-Buffer Overflow in Readu_be	Fixed
LOW	Low-Incorrect Null Pointer Check in SM2 SetPubkey	Fixed

Severity	Findings Title	Status
INFO	Info-The Function MatchTopic argument topic is useless in this function. It will always return true	Fixed
INFO	Info-Useless function	Fixed
INFO	Info-Unused Parameter in function MerkleTree::Root()	Acknowledge
INFO	Info-Replace hard-coded value	Fixed
INFO	Info-Incomplete Log Information in StartupImpl	Fixed
INFO	Info-NeedBypassFork Unnecessary Call in Production Environment	Fixed
INFO	Info-Gas-Validator Struct Fields Are Not Packed	Fixed
INFO	Info-Gas-Optimizing nested loop code	Fixed
INFO	Info-Gas Opt-Inefficient Array Traversal in advanceEpoch Function	Fixed
INFO	Info-std::cout Error Output Stream Inconsistency	Fixed
INFO	Info-Gas-Validator Struct Fields Are Not Packed	Fixed
INFO	Info-Gas-Optimizing nested loop code	Fixed
INFO	Info-Gas Opt-Inefficient Array Traversal in advanceEpoch Function	Fixed
INFO	Info-Inconsistent Module Name in Error Log	Fixed
INFO	Info-Incorrect Service Name in Process Lock Error Message	Fixed
INFO	Info-Invalid Conditional Statement in EVMtxGenerator::GenerateTransaction	Fixed
INFO	Info-Incorrect Error Codes in Ciphertext Parsing Functions	Fixed

Table 2.3: Key Audit Findings

3. DETAILED DESCRIPTION OF FINDINGS

3.1 Critical-Integer Underflow in Gas Calculation

SEVERITY:**CRITICAL****STATUS:****Fixed****DESCRIPTION:**

If `tx_initial_gas()` exceeds the transaction's gas limit, this will cause an integer underflow

No validation prevents negative gas values, allowing free execution

```
int64_t execute_gas_limit = int64_t(tx.GetGas()) - tx_initial_gas(rev, tx);
```

3.2 Critical-Unhandled Exceptions in Modexp::Execute, It will lead to the node DOS

SEVERITY:**CRITICAL****STATUS:****Fixed**

DESCRIPTION:

The `Modexp::Execute` function catches standard exceptions but uses `ALDABA_ASSERT(false)` for newer spec versions. In a release build (where asserts are disabled), this will cause the exception to go unhandled, crashing the node.

```
// ...existing code...
try {
    result = mod == 0 ? bigint(0) : boost::multiprecision::powm(base,
exp_, mod);
} catch (const std::exception& e) {
    VMF_WARN('modexp contract execute failed, exception: {}',
e.what());
    if (spec_version_ < SPEC_VERSION::SPEC_VERSION_V3) {
        return false;
    } else {
        ALDABA_ASSERT(false); // VULNERABILITY: Unhandled exception
in release builds
    }
}
// ...existing code...
```

RECOMMENDATIONS:

The function should gracefully handle the exception and return **false** regardless of the spec version.

3.3 Critical- DoS in Transaction::GetType()

SEVERITY:**CRITICAL****STATUS:****Fixed**

DESCRIPTION:

The function Transaction::GetType() may lead to DOS. It uses `std::visit` to determine the type of a transaction stored in a `std::variant`. The `else` block, intended to handle unexpected transaction types, calls `abort()`.

```
TransactionType Transaction::GetType()const {
    return std::visit(
        [](const auto&tx) ->TransactionType {
            if constexpr (std::is_same_v<std::decay_t<decltype(tx)>,
RawLegacyTransaction>) {
                return TransactionType::eLegacyTxType;
            } else if constexpr (std::is_same_v<std::decay_t<decltype(tx)>,
RawEip2930Transaction>) {
                return TransactionType::eAccessListTxType;

            } else if constexpr (std::is_same_v<std::decay_t<decltype(tx)>,
RawEip1559Transaction>) {
                return TransactionType::eDynamicFeeTxType;
            } else {
                LOGF_FATAL('Unknown tx type');
                abort();
            }
        },
        tx_);
}
```

RECOMMENDATIONS:

Replace the `abort()` call with a robust error-handling mechanism. The function should either throw a catchable exception or return a specific error value .

3.4 Critical- Use of Unvalidated Data After Failed Deserialization in FromSSZ

SEVERITY:**CRITICAL****STATUS:****Fixed****DESCRIPTION:**

The `FromSSZ` function uses `assert` to check the return code of the `ssz::SSZObjectView::Unpack` operation. In release builds, `assert` statements are compiled out. If `Unpack` fails due to malformed or malicious SSZ data, the code will proceed to use the uninitialized `schema_tx` object, which contains garbage data. This leads to undefined behavior, memory corruption, and is highly likely to crash the node.

```
// ...existing code...
    case TransactionType::eLegacyTxType: {
        const ssz::SSZObjectView<schema::LegacyTransaction>
inner_tx_view{
            ssz::Buffer{const_cast<uint8_t*>(tx.ssz_tx.Data()),
tx.ssz_tx.Size()}};
            schema::LegacyTransaction schema_tx;
            auto code = inner_tx_view.Unpack(schema_tx, 0);
            assert(code == ssz::SSZError::OK); // VULNERABILITY: Does
nothing in release builds

            RawLegacyTransaction raw_tx;
            raw_tx.nonce_ = schema_tx.nonce; // Uses uninitialized data
            if Unpack fails
// ...existing code...
```

3.5 Critical-Improper Error Handling in Signature Recovery

SEVERITY:**CRITICAL****STATUS:****Fixed****DESCRIPTION:**

The `EthAccount::Sign` method contains a buG in error handling during signature recovery. When the `Secp256k1Recover` call fails, the function:

1. Only logs a warning message
2. Continues to process potentially invalid or corrupted recovery data
3. Always returns `true` regardless of recovery success

```
auto ret = mycrypto::EthHelper::Secp256k1Recover(
    hash,
    v_flag,
    Util::Readu256_be(std::string_view((const char*)ss.r.data(),
    ss.r.size)),
    Util::Readu256_be(std::string_view((const char*)ss.s.data(),
    ss.s.size)),
    precompile_recover_out);

if (ret != mycrypto::Status::OK) {
    LOGF_WARN('recover fail for code: {}, msg: {}', ret.error_code(),
    ret.error_message());
}

std::string recover_address(precompile_recover_out.data(), 20);
LOGF_INFO('recover account address: {}', toHex(recover_address));

return true; // Always returns true regardless of recovery status
```

The private key is stored in a local variable `privkey` in the `GenerateKey` function but is never explicitly zeroed out or cleared from memory after use:

```
/* static */ Status Bls12_381SignerV1::GenerateKey(Keypair& keypair) {
    // ...
    blsSecretKey privkey;
    // ...
    {
        RLockGuard gu(bls_init_lock_);
```

```
        blsSecretKeySetByCSPRNG(&privkey);  
        // ... use the key ...  
    }  
    // Function returns without clearing privkey  
    return Status::OK;  
}
```

IMPACT:

This vulnerability could lead to:

1. Silent security failures where signature verification appears successful but actually failed
2. Use of invalid or corrupt data in downstream processes

RECOMMENDATIONS:

The function should properly propagate errors by returning **false** when recovery fails and avoid using potentially corrupt data:

```
if (ret != mycrypto::Status::OK) {  
    LOGF_WARN('recover fail for code: {}, msg: {}', ret.error_code(),  
        ret.error_message());  
    return false;  
}
```

3.6 Critical-Insecure Key Generation in SetKey

SEVERITY:**CRITICAL****STATUS:****Fixed****DESCRIPTION:**

The key generation logic is fundamentally broken and results in an extremely weak, 1-byte key instead of the intended 16-byte (128-bit) key.

```
// ...existing code...
Status OsslAesGcmCipherV1::SetKey(std::string& key) {
    if (key.empty()) {
        std::vector<unsigned char> rand_bytes(kDefaultKeyLen);
        RAND_bytes(rand_bytes.data(), static_cast<int>(kDefaultKeyLen));
        key.resize(kDefaultKeyLen);

        key = *(reinterpret_cast<char*>(rand_bytes.data())); //----->>
        BUG: only return 1 bytes of the data
    }
    // ...existing code...
```

The line `key = *(reinterpret_cast<char*>(rand_bytes.data()));` dereferences the pointer to the random data, which gets a single **char**. The `std::string` assignment then creates a string containing only that one character. The effective key space is reduced from 2^{128} to 2^8 , making any encryption trivial to brute-force.

3.7 Critical-Empty Keypair Accepted, Leading to Uninitialized Signer Object

SEVERITY:**CRITICAL****STATUS:****Fixed**

DESCRIPTION:

The `OsslEccK1SignerV1::SetKeypair` function incorrectly handles the case where the input `Keypair` object contains both an empty `privkey_id_` and an empty `pubkey_id_`. Instead of returning an error to indicate that no valid key material was provided, the function does nothing and returns `Status::OK`.

Code Analysis

```
// ...
Status OsslEccK1SignerV1::SetKeypair(const Keypair& keypair) {
    if (KeyType::KEY_ECCK1_PKCS8 != keypair.type_) {
        return STATUS(ErrorCode::CRYPTO_INVALID_KEYTYPE, 'INVALID
KEYTYPE');
    }

    Status ms;
    if (!keypair.privkey_id_.empty()) {
        ms = SetPrivkey(keypair.privkey_id_);
        if (!ms.ok()) {
            return ms;
        }
    }

    if (!keypair.pubkey_id_.empty()) {
        ms = SetPubkey(keypair.pubkey_id_);
        if (!ms.ok()) {
            return ms;
        }
    }
    return Status::OK;
}
```

The logic checks if `privkey_id_` is non-empty and then separately checks if `pubkey_id_` is non-empty. If both are empty, neither of the `if` blocks is entered, and the function proceeds to successfully return `Status::OK`, leaving the `OsslEccK1SignerV1` object in its default, uninitialized state.

3.8 Critical-Denial of Service in Utils::MultiVerify

SEVERITY:**CRITICAL****STATUS:****Fixed**

DESCRIPTION:

The `Utils::MultiVerify` function, which is responsible for verifying multiple signatures against a list of public keys, contains critical design flaws that expose the system to a Denial of Service (DoS) attack. A remote, unauthenticated attacker can craft a small, inexpensive request that forces the server to perform a disproportionately large amount of computationally expensive cryptographic operations, leading to CPU exhaustion and making the service unavailable to legitimate users.

```
// ...
} else { /* multi-sig */
    // bug 1: This loop hashes every public key, creating a large upfront
    // cost.
    for (int pubkey_index = 0; pubkey_index <
        static_cast<int>(pubkey_list.size());
        ++pubkey_index) {
        auto& pubkey = pubkey_list[pubkey_index];
        std::string pubkey_hash(Digester::Digest(pubkey,
            DigestType::SHA256));
        std::string pubkey_fp(pubkey_hash.begin(),
            pubkey_hash.begin() +
            PkeySignature::PUBKEY_FINGER_PRINT_SIZE);
        auto& fp_pubkey_set = pubkey_map[pubkey_fp];
        // ... logic to populate map ...
    }

    for (int index = 0; index < static_cast<int>(signature_list.size());
        ++index) {
        auto& signature = signature_list[index];
        if (signature.size() > PkeySignature::RAW_SIGN_OFFSET) {
            PkeySignature sig_st;
            ms = sig_st.FromBytes(signature);
            // ...
            auto it = pubkey_map.find(sig_st.pubkeyfp_);
            if (it != pubkey_map.end()) {
                // bug 2: This loop iterates over ALL keys with a
                // matching fingerprint.
                for (auto& pubkey_pair : (*it).second) {
```

```
        // An expensive verification is called for each key.
        ms = Utils::DecodeVerify(pubkey_pair.first,
signature, msg);
        if (ms.ok()) {
            pubkey_matched_index.insert(pubkey_pair.second);
            break; // The loop only breaks on SUCCESS.
        } else {
            // ... log error ...
        }
    }
}
} /* for */
}
// ...
```

An attacker can exploit this with the following steps:

1. **Craft Collisions:** The attacker generates a large number such as 10000 of unique public keys that all produce the same truncated hash fingerprint. This is computationally feasible.
2. **Construct Malicious Request:** The attacker submits a request to a system endpoint that calls `MultiVerify`, providing:
 - `pubkey_list`: The list of 1,000 public keys with the colliding fingerprint.
 - `signature_list`: A **single**, invalid signature that is crafted to contain the colliding fingerprint.
3. **Trigger Amplification:** execute the `MultiVerify` function:
 - It first performs 1,000 hash operations to build the `pubkey_map`.
 - It then processes the single signature, looks up its fingerprint, and finds the 1,000 public keys supplied by the attacker.
 - The inner **for** loop begins. It calls the expensive `Utils::DecodeVerify` on the first public key. The verification fails.
 - Because the verification failed, the **break** statement is not reached. The loop continues, attempting to verify the *same invalid signature* against the other 999 public keys.

IMPACT:

The attacker forces the server to perform **10,000 full signature verifications** with a request that contains only **one signature**. This represents a 10000x amplification of the attacker's work. By sending a small number of such requests, the attacker can easily exhaust the server's CPU resources, leading to a **Denial of Service** for all other users of the system

3.9 Critical-Race Condition Leading to Use-After-Free in SelServer

SEVERITY:**CRITICAL****STATUS:****Fixed**

DESCRIPTION:

The `SelServer` function reads from the shared `servers_` vector without acquiring the mutex that protects it. This creates a critical race condition.

Vulnerable Code:

```
// ...existing code...
arpc::EasyRPCChannel* ComboChannel::SelServer(const std::string endpoint)
{
    // BUG: No lock is held here.
    for (auto& srv : servers_) {
        if (srv->endpoint_ == endpoint) {
            return srv->SelectChannel();
        }
    }

    return nullptr;
}

uint32_t ComboChannel::DelServer(const std::string endpoint) {
    // ...
    std::lock_guard<std::mutex> g(lock_); // Lock is held here.

    servers_.erase(remove_if(servers_.begin(),
// ...existing code...
```

Attack Scenario:

1. **Thread A** calls `SelServer` and begins iterating through the `servers_` vector.
2. **Thread B** calls `DelServer` with the same endpoint. It acquires the lock, finds the server object in `servers_`, and erases it. The `unique_ptr` for that server is destroyed, and its memory is deallocated.
3. **Thread A's** loop continues. Its iterator (`srv`) is now a dangling pointer, pointing to the memory that was just freed by Thread B.
4. When Thread A dereferences `srv` to access `srv->endpoint_`, it results in a **use-after-free**.

IMPACT:

This vulnerability can be exploited to cause a denial of service (crash) or, in more sophisticated attacks, could potentially lead to arbitrary code execution.

3.10 Critical-Buffer Bounds Check Removed in release build

SEVERITY:**CRITICAL****STATUS:****Fixed****DESCRIPTION:**

The code uses `assert()` for buffer bounds checking. In release builds, `assert` statements are completely removed by the preprocessor, eliminating this critical security check. An attacker could exploit this by providing a carefully crafted data pointer that points outside the buffer's bounds, causing out-of-bounds memory access.

```
rpc::Cleanup* rpc::AddBuffer(arpc::EasyFastMessage& message,
                             ABuffer&& buf,
                             uint8_t* data,
                             size_t size,
                             bool auto_cleanup) {
    // data,size refer to part of buf, entire buf is kept for cleanup
    until data is used
    if (data) {
        assert(data >= buf.Data() && data + size <= buf.Data() +
buf.Size());
        message.AppendData(data, size);
    }
    // ...
}
```

3.11 Critical-Unsafe Deserialization in ReceiverFromRlp

SEVERITY: CRITICAL**STATUS:** Fixed**DESCRIPTION:**

The function `ReceiverFromRlp` directly dereferences a pointer from the RLP stream's internal buffer without checking if the buffer is empty.

`in.cur_data().data()` may return `nullptr`.

```
void RawLegacyTransaction::ReceiverFromRlp(RLPStreamIn& in) {  
    if (*(in.cur_data().data()) == 0x80) {  
        h0 address;  
        in >> address;  
    }  
}
```

```
bool IsNull() const {  
    return data_.data() == nullptr || data_.size() == 0;  
}
```

IMPACT:

A malformed RLP stream that causes `in.cur_data().data()` to be empty at this point will crash the application, leading to a Denial of Service (DoS).

3.12 Critical-DOG Service Data Race in Consensus Node List Access

SEVERITY:**CRITICAL****STATUS:****Fixed****PATH:**

aldaba-ng/services/dog/dog_service.cpp

DESCRIPTION:

`consensus_node_list_` is accessed without proper synchronization. `OnDomainUpdate()` modifies the list while `GetForwardTransactionRank()` and `ComputeForwardTarget()` methods read from it concurrently.

```
// aldaba-ng/services/dog/dog_service.cpp

// Write
void DOGService::OnDomainUpdate(uint64_t id, const DomainUpdateEvent&
    event) {
    if (event.domain_state == ContractNodeState::NODE_STATE_NORMAL) {
        consensus_node_list_.clear();
        for (auto& info : consensus_node_map_) {
            consensus_node_list_.push_back(info.second); //
        }
        Unsynchronized write
    }
}

// Read
bool DOGService::GetForwardTransactionRank(...) {
    uint64_t index = hash_num % consensus_node_list_.size(); //
    Unsynchronized read
    for (size_t i = 0; i < 3; i++) {
        auto target = consensus_node_list_[(index + i) %
            consensus_node_list_.size()].node_id_; // Unsynchronized read
    }
}
```

RECOMMENDATIONS:

using read-write locks.

```
+ ReadWriteLock consensus_node_list_lock_;

void DOGService::OnDomainUpdate(uint64_t id, const DomainUpdateEvent&
    event) {
+   WLockGuard _(consensus_node_list_lock_);
    if (event.domain_state == ContractNodeState::NODE_STATE_NORMAL) {
        consensus_node_list_.clear();
        for (auto& info : consensus_node_map_) {
            consensus_node_list_.push_back(info.second);
        }
    }
}

bool DOGService::GetForwardTransactionRank(...) {
+   RLockGuard _(consensus_node_list_lock_);
    uint64_t index = hash_num % consensus_node_list_.size();
    for (size_t i = 0; i < 3; i++) {
        auto target = consensus_node_list_[(index + i) %
            consensus_node_list_.size()].node_id_;
    }
}
```

3.13 Critical-TOCTOU Race Condition in DDosDefender::ForwardInc

SEVERITY:**CRITICAL****STATUS:****Fixed****DESCRIPTION:**

The function performs a non-atomic check-then-increment operation while holding only a **shared read lock**, allowing multiple threads to concurrently pass the rate limit check and increment counters beyond the configured limits.

```
bool DDosDefender::ForwardInc(TxContext* tx) {
    ReadGuard lk(forwarded_tx_throttle_lock_); // SHARED LOCK - Multiple
    threads allowed

    if(forwarded_tx_domains_.find(tx->GetProposer()) ==
    forwarded_tx_domains_.end()) {
        return LimitForwardedTxCountFromFullNode(tx);
    } else {
        auto it = forwarded_tx_count_.find(tx->GetProposer());
        if (it == forwarded_tx_count_.end()) {
            return false;
        }

        // STEP 1: CHECK - Read current counter value
        if (it->second->load() >= forwarded_each_proposer_max_size) {
            return false; // Rate limit check
        }

        // STEP 2: RACE WINDOW - Other threads can execute here
        concurrently

        // STEP 3: USE - Increment without re-validation
        it->second->fetch_add(1); // VULNERABLE: Non-atomic with check
        above
        return true;
    }
}
```

3.14 Critical-Inconsistent Lock Usage in FetchingLock

SEVERITY: CRITICAL**STATUS:** Fixed

DESCRIPTION:

A critical race condition vulnerability exists in `DDosDefender::FetchingLock()` due to inconsistent lock usage across two separate critical sections, allowing potential state corruption and security bypass. The function uses two different locks in sequence with a critical gap between them, allowing the state to change between validations and modifications.

```
bool DDosDefender::FetchingLock(const std::string& fetcher) {
    {
        ReadGuard lk(forwarded_tx_throttle_lock_); // First lock (read)
        // if GetProposer() of tx is in gateway.
        if (forwarded_tx_domains_.find(fetcher) ==
            forwarded_tx_domains_.end()) {
            LOGF_WARN('ho {} in forwarded_tx_domains_', fetcher);
            return false;
        }
    } //-----> Bug First lock released HERE - RACE WINDOW BEGINS

    {
        WriteGuard lk(fetching_lock_); // Second lock (write) - RACE
        WINDOW ENDS
        if (fetching_domains_.find(fetcher) != fetching_domains_.end()) {
            LOGF_WARN('{} is locking in forwarded_tx_domains_', fetcher);
            return false;
        }
        fetching_domains_.emplace(fetcher);
    }
    return true;
}
```

SCENARIO:

Sequence of Events

1. Thread A: Calls `FetchingLock('domain1')`
2. Thread A: Acquires read lock on `forwarded_tx_throttle_lock_`

3. Thread A: Verifies `'domain1'` exists in `forwarded_tx_domains_`
4. Thread A: **Releases** read lock on `forwarded_tx_throttle_lock_`
5. Thread B: Calls `DelDomain('domain1')`
6. Thread B: Acquires write lock on `forwarded_tx_throttle_lock_`
7. Thread B: Removes `'domain1'` from `forwarded_tx_domains_`
8. Thread B: Releases write lock
9. Thread A: Acquires write lock on `fetching_lock_`
10. Thread A: Adds `'domain1'` to `fetching_domains_` *despite it no longer being valid*

3.15 Critical-Race Condition in Rule State Updates

SEVERITY:**CRITICAL****STATUS:****Fixed****DESCRIPTION:**

A critical race condition vulnerability exists in the `ERSManager::RuleStateUpdate` function where rule state modifications are performed without proper synchronization, leading to potential data corruption and inconsistent system states.

The `RuleStateUpdate` function contains a race condition where the rule's state is modified outside of any synchronization mechanism:

```
bool ERSManager::RuleStateUpdate(uint64_t id, RuleState state) {
    // ... validation code ...

    InferRuleSharedPtr rule{nullptr};
    {
        std::shared_lock _(rw_lock_); // Shared lock for reading
        auto it = id_index_.find(id);
        if (it != id_index_.end()) {
            rule = it->second;
        }
    } // Lock released here

    // ... validation code ...

    RuleState old_state = rule->state; // Unprotected read
    rule->state = state;               // Unprotected write - RACE
    CONDITION

    // ... logging code ...
}
```

3.16 Critical- Race Condition in Bloom Filter Update

SEVERITY:**CRITICAL****STATUS:****Fixed**

DESCRIPTION:

In `AddInferRule`, the rule is added to the `infer_rules_ map` under a lock, but the bloom filter is updated (`bloom_filter_.Add(rule_filter);`) *after* the lock is released.

This creates a time window where another thread in `MatchRule` can check the bloom filter, see the rule is not present (because the update hasn't happened yet), and skip checking the map, thus missing a valid, newly-added rule.

```
bool ERSManager::AddInferRule(InferRuleSharedPtr rule) {
    if (!rule || !rule->id) {
        LOG_KV_ERROR(sLogger, ('AddInferRule failed', 'invalid rule'));
        return false;
    }

    if ((rule->contract_addr.empty() && rule->metahash.empty()) ||
        rule->selector.empty()) {
        LOG_KV_ERROR(sLogger,
                     ('AddInferRule failed', 'invalid rule')
                     ('RuleInfo', rule->ToJsonString()));
        return false;
    }

    std::string rule_filter;
    if (rule->contract_addr.empty()) {
        rule_filter = rule->metahash + rule->selector;
    } else {
        rule_filter = rule->contract_addr + rule->selector;
    }

    // insert to infer rules
    {
        std::unique_lock _(rw_lock_);
        infer_rules_[rule_filter] = rule;
        id_index_[rule->id] = rule;
    }

    // update to bloomfilter
```

```
bloom_filter_.Add(rule_filter); //---->BUG the lock released
```

```
InferType ERSManager::MatchRule(TxContextSharedPtr tx_ctx,
    InferRuleSharedPtr& rule) {
    if (tx_ctx->to_acc_ && tx_ctx->to_acc_->IsEOA()) {
        return InferType::EVM_TRANSFER;
    }

    // eoa account create by transfer to
    if (!tx_ctx->to_acc_ && !tx_ctx->GetTo().empty() &&
        tx_ctx->GetSlector().empty()) {
        return InferType::EVM_TRANSFER;
    }
    do {
        if (tx_ctx->to_acc_ && !tx_ctx->to_acc_->IsEOA()) {
            auto contract = tx_ctx->GetTo();
            auto metahash = tx_ctx->GetMetaHash();
            auto selector = tx_ctx->GetSlector();

            std::string rule_filter = contract + selector;
            if (!bloom_filter_.Contains(rule_filter)) { //----->
                because the bug. in here it will be step into.
                if (metahash.empty()) {
                    break;
                }
                rule_filter = metahash + selector;
                if (!bloom_filter_.Contains(rule_filter)) {
                    break;
                }
            }
        }
    }
```


3.17 Critical-Unauthorized Access to Debug APIs Allows Remote Configuration Modification

SEVERITY:**CRITICAL****STATUS:****Fixed****PATH:**`aldaba-ng/services/portal/dispatcher.cpp`**DESCRIPTION:**

`debug_*` RPC are exposed without proper authentication or authorization controls, allowing any remote attacker to modify critical runtime configurations including security-critical flags.

`InstallDebugRouter()` debug endpoints are registered with only basic format validation middleware, completely lacking authentication checks.

```
// aldaba-ng/services/portal/dispatcher.cpp
bool Dispatcher::InstallDebugRouter(std::shared_ptr<lightnet::Router>
    router,

    std::shared_ptr<lightnet::ContextManager> ctx_manager) {
    std::vector<lightnet::HandlerFunc> middlewares{[this](auto ctx) {
        Metric(ctx);
    },
        [this](auto ctx) {
            ETHVerify(ctx);
        }
    };
    auto workers = ctx_manager->GetDebugWorker().get();

    auto set_global_flag_handler = [this](auto ctx) {
        DebugSetGlobalFlag(ctx);
    };
    router->Route('debug_setGlobalFlag', 0, set_global_flag_handler,
        middlewares, workers);
    // ...
}
```

`ETHVerify` middleware only performs basic JSON-RPC format validation without any authentication.

```
// aldaba-ng/services/portal/dispatcher.cpp
void Dispatcher::ETHVerify(std::shared_ptr<lightnet::Context> ctx) {
    if (ctx->GetRpcType() != 'jsonrpc') {
        ServeError(ctx, JSONRPCError::INVALID_REQUEST, '
UNSUPPORTED_RPC_TYPE');
        return;
    }
    auto jsonrpc_ctx =
std::dynamic_pointer_cast<lightnet::JSONRPCContext>(ctx);
    auto jsonrpc_request = jsonrpc_ctx->GetRequest();
    if (!jsonrpc_request->params.isArray() &&
!jsonrpc_request->params.isNull()) {
        ServeError(ctx, JSONRPCError::INVALID_REQUEST, '
PARAM_IS_NOT_ARRAY');
        return;
    }
    ctx->Next(); // No authentication check performed
}
```

`DebugSetGlobalFlag` handler directly modifies global runtime flags without authorization.

```
// aldaba-ng/services/portal/dispatcher.cpp
void Dispatcher::DebugSetGlobalFlag(std::shared_ptr<lightnet::Context>
ctx) {
    // ...

    auto all_flags = util::GetGlobalFlags();
    std::map<std::string, std::string> accepts;
    for (auto& [set_key, set_value] : set_flags) {
        auto it = all_flags.find(set_key);
        if (it == all_flags.end()) {
            ServeError(ctx, JSONRPCError::INVALID_PARAMS, '
PARAM_VERIFY_ERROR: unknown flag name');
            return;
        }
        accepts[set_key] = set_value;
    }
    // warning: current only work in lite mode
    util::SetGlobalFlags(accepts); // Direct modification without auth
    check

    Json::Value data = 'success';
}
```

```
ServeETHData(ctx, data);  
}
```

Attack example:

```
{  
  'jsonrpc': '2.0',  
  'method': 'debug_setGlobalFlag',  
  'params': [{ 'enable_eip155': 'false' }],  
  'id': 1  
}
```

RECOMMENDATIONS:

Disable debug APIs in production via configuration flag.

3.18 High-Private Key Exposure in Logs

SEVERITY:**HIGH****STATUS:****Fixed****DESCRIPTION:**

The `EthAccount` class in `evm_account.cpp` is logging sensitive cryptographic material (private keys) as part of standard INFO-level logging. This occurs in both the `ExtendSecret()` method and the `Sign()` method, where the following logging statement appears:

```
LOGF_INFO('secret key: {}, public key: {}, address: {}',
          toHex(std::string(reinterpret_cast<const char*>(secret_key_),
                           SECRET_KEY_LEN)),
          toHex(std::string(reinterpret_cast<const char*>(public_key_),
                           PUBLIC_KEY_LEN)),
          toHex(std::string(reinterpret_cast<const char*>(address_),
                           ADDRESS_LENGTH)));
```

IMPACT:

- **Critical exposure:** Private keys (`secret_key_`) should never be logged as they provide complete account control
- **Data leak:** Logs may be accessible by unauthorized personnel or stored in unsecured locations

RECOMMENDATIONS:

1. Remove all instances of logging the secret key

3.19 High-State Inconsistency in Withdrawal Mechanism Leads to Permanent Fund Loss

SEVERITY:**HIGH****STATUS:****Fixed****PATH:**

aldaba-ng/sys_contract/artifact_src/solidity/sys_staking.sol

DESCRIPTION:

In `withdrawStake()` function, `validators[_poolId].totalStake` is reduced by `_withdrawStake` amount immediately. Later in `advanceEpoch()` function, the validator's `pendingWithdrawStake` is cleared before confirming successful transfer. The `_transferTo()` function only emits error events without reverting to transfer failures, allowing state updates to persist despite failed transfers.

```
// aldaba-ng/sys_contract/artifact_src/solidity/sys_staking.sol

// withdrawStake()
validators[_poolId].pendingWithdrawStake = _withdrawStake;
validators[_poolId].totalStake -= _withdrawStake; // totalStake reduced

// advanceEpoch()
while (pending_withdraw_index < pendingExitPoolIds.length) {
    bytes32 poolId = pendingExitPoolIds[pending_withdraw_index];
    if (validators[poolId].pendingWithdrawWindow == 0) {
        uint pending_withdraw_stake =
            validators[poolId].pendingWithdrawStake;
        validators[poolId].pendingWithdrawStake = 0; // State cleared
        first
        _transferTo(validators[poolId].owner, pending_withdraw_stake);
        // Transfer attempted after
        // ... array manipulation
    }
}

function _transferTo(address recipient, uint amount) internal {
    if (amount > address(this).balance) {
        emit ErrorOccurred(...);
        return; // No revert, execution continues
    }
}
```

```
    }
    (bool success, ) = recipient.call{value: amount}('');
    if (!success) {
        emit ErrorOccurred(...);
        return; // No revert, execution continues
    }
}
```

Redundant $O(n^2)$ array search operations in validator status processing causing excessive gas consumption during epoch advancement. When validator status is already set to 0 and `pendingWithdrawWindow != 0`, the next epoch's `advanceEpoch` still traverses `activePoolIds` to find deactivated validators.

```
// aldaba-ng/sys_contract/artifact_src/solidity/sys_staking.sol

// Process pending exits - searches activePoolIds even for deactivated
// validators
for (uint i = 0; i < pendingExitPoolIds.length; i++) {
    bytes32 poolId = pendingExitPoolIds[i];
    for (uint j = 0; j < activePoolIds.length; j++) {
        if (activePoolIds[j] == poolId) {
            if (validators[poolId].totalStake < MIN_POOL_STAKE) {
                validators[poolId].status = 0; // Deactivated
                activePoolIds.pop();
            }
            break;
        }
    }
}
```

IMPACT:

Validators permanently lose their stake funds when transfers fail due to insufficient contract balance or recipient rejection, as the withdrawal state is irreversibly cleared while funds remain in the contract.

RECOMMENDATIONS:

Modify `_transferTo()` to return transfer status and handle failures appropriately.

3.20 High-Invalid Address validation

SEVERITY:**HIGH****STATUS:****Fixed****PATH:**

aldaba-ng/libraries/structures/evm/raw/raw_eip_1559_transaction.cpp

DESCRIPTION:

Code path: aldaba-ng/libraries/structures/evm/raw/raw_eip_1559_transaction.cpp

in the `ReceiverFromJson`:

```
if (to.size() >= 2 && to.substr(0, 2) == '0x') {
    to_ = EthAddress::fromHexToArrayHelper(to);
} else {
    to_ = EthAddress(to);
}
```

The code only checks that `to` has at least 2 characters and starts with “0x” prefix, There is no validation of the actual address content or length, If `to` is exactly “0x” (with no following hex digits), it will. Pass the validation check, Call `EthAddress::fromHexToArrayHelper(to)`, Attempt to read non-existent hex data, Result in an out-of-bounds read.

3.21 High-Missing Error Handling, may lead to the node crash

SEVERITY:**HIGH****STATUS:****Fixed****DESCRIPTION:**

The code doesn't check if `OpenChannel` fails and returns NULL. Then dereference the channel which will lead to NULL dereference, which will lead to the node crash.

```
arpc::EasyRPCChannel* channel =
    io_>OpenChannel(s->endpoint_, ERPC_CONN_IDLE_TIME, false, 1024); //
    Line 28
// No check if channel is NULL
arpc::ChannelOption option;
    option.policy = 'fastpath';
    channel->Init(option);
    channel->SetPeerThread(i, s->server_io_thread_num_);
    channel->SetConnIdleTime(60 * 60 * 60 * 1000);
    channel->SetTraceFlag(true);
```


3.22 High-State Inconsistency in Withdrawal Mechanism Leads to Permanent Fund Loss

SEVERITY:**HIGH****STATUS:****Fixed****PATH:**

aldaba-ng/sys_contract/artifact_src/solidity/sys_staking.sol

DESCRIPTION:

In `aldaba-ng/sys_contract/artifact_src/solidity/sys_staking.sol::withdrawStake()` function, `validators[_poolId].totalStake` is reduced by `_withdrawStake` amount immediately. Later in `advanceEpoch()` function, the validator's `pendingWithdrawStake` is cleared before confirming successful transfer. The `_transferTo()` function only emits error events without reverting on transfer failures, allowing state updates to persist despite failed transfers.

```
// withdrawStake()
function withdrawStake(bytes32 _poolId, uint256 _withdrawStake)
external {
    require(validators[_poolId].poolId != 0, 'Validator does not exist');
    require(validators[_poolId].owner == msg.sender, 'Validator does not exist');
    require(!isValidatorPendingExit(_poolId), 'Validator is pending exit');
    require(validators[_poolId].totalStake >= _withdrawStake, 'WithdrawStake is too large');

    validators[_poolId].pendingWithdrawStake = _withdrawStake;
    validators[_poolId].totalStake -= _withdrawStake; // totalStake reduced
    validators[_poolId].pendingWithdrawWindow =
getWithdrawEffectiveWindow();

    pendingExitPoolIds.push(_poolId);
```

```
        emit ValidatorWithdrawStake(_poolId,
            currentEpoch,
            block.number,
            validators[_poolId].totalStake,
            _withdrawStake
        );
    }

// advanceEpoch()
function advanceEpoch(bytes32[] memory _poolIds, uint256[] memory
_priority_fees) public onlyOwner {
    // ...
    uint pending_withdraw_index = 0;
    while (pending_withdraw_index < pendingExitPoolIds.length) {
        bytes32 poolId = pendingExitPoolIds[pending_withdraw_index];
        if (validators[poolId].pendingWithdrawWindow == 0) {
            // effective withdraw window is reached
            uint pending_withdraw_stake =
validators[poolId].pendingWithdrawStake;
            validators[poolId].pendingWithdrawStake = 0;
            _transferTo(validators[poolId].owner,
pending_withdraw_stake); // transfer without error handling
            pendingExitPoolIds[pending_withdraw_index] =
pendingExitPoolIds[pendingExitPoolIds.length - 1];
            pendingExitPoolIds.pop();
        } else {
            validators[poolId].pendingWithdrawWindow -= 1;
            pending_withdraw_index++;
        }
    }
}

// _transferTo()
function _transferTo(address recipient, uint amount) internal {
    if (amount > address(this).balance) { // emit error event but
not revert
        uint error_code = 1;
        bytes memory errorData = abi.encode(recipient, amount,
address(this).balance);
        emit ErrorOccurred(
            currentEpoch,
            block.number,
            error_code,
```

```

        errorData
    );
    return;
}

(bool success, ) = recipient.call{value: amount}('');

if (!success) { // emit error event but not revert
    uint error_code = 2;
    bytes memory errorData = abi.encode(recipient, amount,
address(this).balance);
    emit ErrorOccurred(
        currentEpoch,
        block.number,
        error_code,
        errorData
    );
    return;
}
}

```

Meanwhile, in [aldaba-ng/services/controller/controller_service.cpp](#), it specifically listens to and processes emitted events such as **DomainUpdate** and **EpochChange**, but does not listen to or handle **ErrorOccurred**.

```

void ControllerService::OnDomainUpdate(const DomainUpdateEvent& event) {
    // using Digest(pubkey) as node_id instead of that defined in
    sys_contract
    // NodeId node_id = fromHexToString(event.domain_id);
    PublicKey pubkey = event.pubkey;
    StabilizePublicKey stabilizing_pubkey;
    auto ret = stabilizing_pubkey.FromBytes(event.stabilizing_pubkey);
    NodeId node_id = mycrypto::Digester::Digest(pubkey).substr(0,
NODE_ID_LENGTH);
    NodeRole node_role = static_cast<NodeRole>(event.domain_role);
    BlockNumber block_num = event.block_num;
    BlockNumber effective_block_num = event.effective_block_num;
    if (!ret.ok()) {
        LOGF_ERROR(
            'Invalid aggr pubkey, node id: {}, pubkey: {}, aggr pubkey:
    {}, block_num: {}, '
            'effective_block_num: {}',

```

```
        event.domain_id,
        pubkey,
        event.stabilizing_pubkey,
        block_num,
        effective_block_num);
    }

    LOGF_INFO(
        'Controller receive DomainUpdate event. Node_id is [{}], pubkey
is [{}], '
        'stabilizing_pubkey '
        'is '
        '[{}], update_type is [{}], block_num is [{}],
effective_block_num is [{}]',
        event.domain_id,
        toHex(pubkey),
        toHex(stabilizing_pubkey.GetPublicKey()),
        event.domain_state,
        block_num,
        effective_block_num);

    // only update consensus role
    if (node_role == NodeRole::NODE_ROLE_CONSENSUS) {
        if (event.domain_state == ContractNodeState::NODE_STATE_NORMAL) {
            domain_cache_>ChangeValidators(effective_block_num,
                                           node_id,
                                           pubkey,
                                           stabilizing_pubkey);

        } else if (event.domain_state ==
ContractNodeState::NODE_STATE_DELETED) {
            domain_cache_>DeleteValidators(effective_block_num, node_id);
        } else {
            LOGF_WARN('receive unknown DomainUpdateEvent type {},
return', event.domain_state);
            return;
        }
    } else {
        LOGF_INFO('Update non-consensus node. pass');
        return;
    }

    if (synchronizer_ != nullptr) {
        synchronizer_>OnEventDomainChanged(event);
    }
}
```

```
    }
    if (IsLegacyStabilizer()) {
        std::lock_guard<std::mutex> lock(stabilizer_mutex_);
        if (block_stabilizer_ != nullptr) {
            block_stabilizer_>OnEventDomainChanged(event);
        }
    }
}
```

IMPACT:

Validators permanently lose their staked funds when transfers fail due to insufficient contract balance or recipient rejection, as the withdrawal state is irreversibly cleared while funds remain in the contract.

RECOMMENDATIONS:

Modify `_transferTo()` to return transfer status and handle failures appropriately Or listen and handle `ErrorOccurred` event in `controller_service.cpp`.

3.23 High-Hardcoded Password in Cryptographic Key Operations

SEVERITY:**HIGH****STATUS:****Fixed****PATH:**

aldaba-ng/services/controller/helper.hpp\aldaba-ng/services/dog/dog_helper.cpp\
scripts/aldaba-ops/aldaba_ops/toolkit/config.py\

DESCRIPTION:

Multiple components in the codebase use hardcoded passwords for cryptographic key operations, creating a serious security vulnerability. The same weak password 123abc is consistently used across different modules for loading domain keys.

```
// aldaba-ng/services/controller/helper.hpp

virtual bool LoadKeypair(const std::string& key_js_str,
    mycrypto::Keypair& key_pair) {
    // ...
    std::string key_passwd = '123abc'; // FIXME: hardcode key passwd
    mycrypto::KeyOperatorPtr op = mycrypto::Utils::GetKeyOperator();
    auto ms = op->Load(key_file, key_passwd, key_pair);
    // ...
}
```

```
// aldaba-ng/services/dog/dog_helper.cpp

bool DOGHelper::WriteDomainKey(const std::string& domain_name,
    const std::string& key_js_str,
    mycrypto::Keypair& key_pair) {

    // ...
    std::string key_passwd = '123abc'; // FIXME: read from config_
    mycrypto::KeyOperatorPtr op = mycrypto::Utils::GetKeyOperator();
    auto ms = op->Load(key_file, key_passwd, key_pair);
    // ...
}
```

```
// scripts/aldaba-ops/aldaba_ops/toolkit/config.py
```

```
@dataclass
class DomainSecretConfig(object):
    key_passwd: str = field(default='123abc', metadata={'required':
False})

@dataclass
class ClientSecretConfig(object):
    key_passwd: str = field(default='123abc', metadata={'required':
False})
```

- FIXME comments indicate awareness of the issue and plans for future improvement

```
// helper.hpp
auto tmp_secret_path = fs::temp_directory_path() /
    config->GetDomainName() / 'controller/secrets/';
fs::create_directories(tmp_secret_path);
std::string key_file = tmp_secret_path.string() + 'domain.key';
```

- Keys are written to predictable paths in temporary directories

RECOMMENDATIONS:

Remove Hardcoded Passwords:

```
- std::string key_passwd = '123abc'; // FIXME: hardcode key passwd
+ std::string key_passwd;
+ if (!config->GetValue('domain_key_password', &key_passwd)) {
+     LOGF_ERROR('Domain key password not configured');
+     return false;
+ }
```

3.24 High-DOG Service Timer Cancellation Failure Leading to Post-Shutdown Execution

SEVERITY:**HIGH****STATUS:****Fixed****PATH:**`aldaba-ng/services/dog/dog_service.cpp\aldaba-ng/libraries/utils/timer.cpp\`**DESCRIPTION:**

Timer callbacks in `DOGService` ignore cancellation signals by not checking `error_code`, causing timers to restart after `Cancel()` is called during shutdown.

```
// aldaba-ng/services/dog/dog_service.cpp

void DOGService::ShutdownImpl() {
    if (forward_check_timer_ != nullptr) {
        forward_check_timer_>Cancel();
    }
    if (conn_check_timer_ != nullptr) {
        conn_check_timer_>Cancel();
    }
    // ...
}

void DOGService::OnCheckConn() {
    // ...

    conn_check_timer_>ExpiresAfter(std::chrono::milliseconds(connection_check_interval_));
    conn_check_timer_>AsyncWait(std::bind(&DOGService::OnCheckConn,
    this)); // Restart
}

void DOGService::OnCheckForward() {
    // ...

    forward_check_timer_>ExpiresAfter(std::chrono::milliseconds(forward_check_interval_));

    forward_check_timer_>AsyncWait(std::bind(&DOGService::OnCheckForward,
    this)); // Restart
}
```



```
}
```

```
// aldaba-ng/libraries/utils/timer.cpp

// Timer expects callback with error_code parameter
void Timer::AsyncWait(std::function<void(const std::error_code&)>
    handler) {
    timer_.async_wait(handler);
}
```

RECOMMENDATIONS:

Modify callback signatures to accept error_code and check for cancellation.

```
// dog_service.h
- void OnCheckConn();
- void OnCheckForward();
+ void OnCheckConn(const std::error_code& ec);
+ void OnCheckForward(const std::error_code& ec);

// dog_service.cpp
- void DOGService::OnCheckConn() {
+ void DOGService::OnCheckConn(const std::error_code& ec) {
+   if (ec == boost::asio::error::operation_aborted) {
+     return;
+   }
+   // ...

  conn_check_timer_->ExpiresAfter(std::chrono::milliseconds(connection_check_interval));
-   conn_check_timer_->AsyncWait(std::bind(&DOGService::OnCheckConn,
+   conn_check_timer_->AsyncWait(std::bind(&DOGService::OnCheckConn,
+   this, std::placeholders::_1));
}

- void DOGService::OnCheckForward() {
+ void DOGService::OnCheckForward(const std::error_code& ec) {
+   if (ec == boost::asio::error::operation_aborted) {
+     return;
+   }
+   // ...
```

```
forward_check_timer_>ExpiresAfter(std::chrono::milliseconds(forward_check_interval));  
-  
forward_check_timer_>AsyncWait(std::bind(&DOGService::OnCheckForward,  
this));  
+  
forward_check_timer_>AsyncWait(std::bind(&DOGService::OnCheckForward,  
this, std::placeholders::_1));  
}
```

3.25 High-Cache Miss Handling Inadequacy Causes State Desynchronization

SEVERITY:**HIGH****STATUS:****Fixed****PATH:**

aldaba-ng/services/compute/compute_service.cpp

DESCRIPTION:

When processing transaction commit, if the expected execution result cannot be found in the cache, the program silently fails and skips the persistence step. Although there is error checking in `CommitTxGroupDone`, the error state is not properly set.

```
// aldaba-ng/services/compute/compute_service.cpp
void ComputeService::CommitTxGroup(CUExecuteRPCContextRawPtr
    exec_context) {
    CETINA_SCOPE_EXIT_DURATION_NS(TXGROUP_COMMIT_DURATION);
    DEFER(exec_context->callback_());

    // first init state
    exec_context->tx_group_context->InitCurrentState();

    ExecutorTxGroupContextSPtr exec_tx_group_ctx;
    if (!compute_cache->GetContext(exec_context->TaskID(),
        exec_tx_group_ctx)
        || !exec_tx_group_ctx) {
        CU_LOGF_ERROR('compute cache miss task_id: {}, commit tx group
            cause issue, rerun group',
                exec_context->TaskID());
        // force rerun group and commit
        return;
    } else {
        CU_LOGF_DEBUG('compute cache hit task_id: {}',
            exec_context->TaskID());
    }

    exec_context->tx_group_context_ = exec_tx_group_ctx;

    std::vector<std::function<void()>> pipeline = {
        [this, exec_context]() {
```

```
        Persistent(exec_context);
    },
};

for (auto& task : pipeline) {
    task();
    if (exec_context->result_ != ErrorCode::SUCCESS) {
        CU_LOGF_WARN('execute tx group done cause issue, interrupt
process');
        break;
    }
}

// push executed tx result to txpool,
PushExecutedTxResult(exec_context);
}
```

The `CommitTxGroupDone` callback is still triggered due to `DEFER(exec_context->callback())`.

```
// alibaba-ng/services/compute/compute_service.cpp
void ComputeService::CommitTxGroupDone(CUExecuteRPCContextRawPtr
exec_context) {
    CETINA_SCOPE_EXIT_DURATION_NS(TXGROUP_COMMIT_STAGE_RESPONSE_DURATION);
    std::vector<std::function<void()>> pipeline = {
        [this, exec_context]() {
            exec_context->resp_ = GenerateResponse(exec_context, false);
        },
    };

    for (auto& task : pipeline) {
        task();
        if (exec_context->result_ != ErrorCode::SUCCESS) {
            CU_LOGF_WARN('commit tx group done cause issue, interrupt
process');
            break;
        }
    }

    SendResponse(exec_context);

    // Cleanup cache
    if (!compute_cache_->RemoveContext(exec_context->TaskID()))
```

```
        || !compute_cache_->RemoveBlock2Task(exec_context->BlockNumber(),
exec_context->TaskID())) {
        CU_LOGF_DEBUG('remove task_id: {} not exist',
exec_context->TaskID());
    }
}
```

RECOMMENDATIONS:

Set the error state properly when cache miss occurs. Either retry the entire transaction group execution or return an explicit error.

```
void ComputeService::CommitTxGroup(CUExecuteRPCContextRawPtr
exec_context) {
    if (!compute_cache_->GetContext(exec_context->TaskID(),
exec_tx_group_ctx)
        || !exec_tx_group_ctx) {
        CU_LOGF_ERROR('compute cache miss task_id: {}, commit tx group
cause issue, rerun group',
            exec_context->TaskID());
        // force rerun group and commit
+         exec_context->result_ = ErrorCode::COMPUTE_CACHE_MISS; // Set
error state
        return;
    }
}
```

3.26 High-uninitialized objects in function AggregateKeypair

SEVERITY:**HIGH****STATUS:****Fixed****DESCRIPTION:**

in the function `AggregateKeypair`, `in_vec.resize(keypair_vec.size())` creates `N` default-constructed `AggregateKeypair` entries. The subsequent loop parses `N` items and `emplace_back` them, resulting in a vector of size `2N`. The first `N` entries remain default-constructed. Their internal `blsPublicKey (pubkey_)` is not initialized.

```
Status AggregateKeypair::AggregateKeypair(const
    std::vector<std::string>& keypair_vec) {
    std::vector<AggregateKeypair> in_vec;
    in_vec.resize(keypair_vec.size());

    for (const auto& keypair : keypair_vec) {
        AggregateKeypair in;
        auto ms = in.FromBytes(keypair);
        if (!ms.ok()) {
            return STACKED_STATUS(ErrorCode::CRYPTO_AGGREGATION_FAILED,
ms);
        }

        in_vec.emplace_back(std::move(in));
    }

    return AggregateKeypair(in_vec);
}
```

IMPACT:

This will lead to data corruption, or memory-safety issues depending on the BLS library's assumptions.

3.27 High-AggregableKeypair's pubkey guard inverted

SEVERITY:**HIGH****STATUS:****Fixed****DESCRIPTION:**

the code `if (!IsPubkey() || keypair.IsPubkey())` rejects valid inputs where both are pubkeys.

```
Status AggregableKeypair::AggregateKeypair(const AggregableKeypair&
keypair) {
    (void)Bls12_381SignerV1::inited_;

    if (!IsPubkey() || keypair.IsPubkey()) {
        return STATUS(ErrorCode::CRYPTO_AGGREGATION_FAILED, 'ONLY SUPPORT
PUBKEY AGGREGATION');
    }

    {
        RLockGuard gu(*Bls12_381SignerV1::GetRWLock());
        blsPublicKeyAdd(&pubkey_, &keypair.pubkey_);
    }

    UpdatePubkeyIdAfterAggr();

    return Status::OK;
}
```

3.28 High- Private key bytes not securely erased

SEVERITY:**HIGH****STATUS:****Fixed****DESCRIPTION:**

Keypair stores private key material in `privkey_id_` which is a `std::string`. When the keypair is reset or destroyed, `privkey_id_.clear()` is used, which only adjusts size but leaves the sensitive bytes in heap memory until reused. This leaks secret data through heap inspection, crash dumps, paging.

```
class Keypair {
public:
    Keypair();

public:
    virtual Status SetPublicKey(std::string const& in);
    virtual Status SetPrivateKey(std::string const& in);

    Status GetPublicKey(std::string& out) const;
    Status GetPrivateKey(std::string& out) const;
    std::string GetPublicKey() const;
    std::string GetPrivateKey() const;

    bool IsPubkey() const;
    bool IsPrivkey() const;

public:
    virtual Status FromBytes(const std::string_view in);
    Status ToBytes(std::string& out, bool is_public) const;

public:
    KeyType type_ = KeyType::UNKNOWN;
    // for some providers, this is the private key itself, and otherwise
    the
    // identity of private key stored somewhere else
    PrivateKey privkey_id_; //-----
    PublicKey pubkey_id_;
};
```


RECOMMENDATIONS:

Add a destructor that reset `privkey_id_` memory

3.29 High-missing BLS serialization failure handling

SEVERITY:**HIGH****STATUS:****Fixed****DESCRIPTION:**

`AggregableKeypair::FromBytes` calls `blsPublicKeySerialize(buf, kBlsMaxKeySize, &pubkey_)` and copies `n` bytes into `pubkey_id_`.

If the BLS serialization API fails and return 0, the code still treats it as success, leaving `pubkey_id_` empty and `pubkey_` potentially invalid.

`AggregablePkeySignature::ToBytes` does the same with `blsSignatureSerialize`; on failure it emits an empty signature but returns OK.

```
Status AggregableKeypair::FromBytes(const std::string_view in) {
    (void)Bls12_381SignerV1::inited_;

    auto ms = Keypair::FromBytes(in);
    if (!ms.ok()) {
        return ms;
    }

    if (type_ != KeyType::KEY_BLS_12_381) {
        return STATUS(ErrorCode::CRYPTO_INVALID_KEYTYPE, 'INVALID
KEYTYPE ');
    }

    if (!pubkey_id_.empty()) {
        RLockGuard gu(*Bls12_381SignerV1::GetRWLock());
        if (0 == blsPublicKeyDeserialize(&pubkey_, pubkey_id_.data(),
pubkey_id_.size())) {
            return STATUS(ErrorCode::CRYPTO_INVALID_PUBKEY, 'INVALID
PUBKEY ');
        }
    }

    if (!privkey_id_.empty()) {
        char buf[kBlsMaxKeySize];
        size_t n;
        {
            RLockGuard gu(*Bls12_381SignerV1::GetRWLock());
```

```
        if (0 == blsSecretKeyDeserialize(&privkey_,
privkey_id_.data(), privkey_id_.size())) {
            return STATUS(ErrorCode::CRYPTO_INVALID_PRIVKEY, 'INVALID
PRIVKEY ');
        }

        blsGetPublicKey(&pubkey_, &privkey_);

        n = blsPublicKeySerialize(buf, kBlsMaxKeySize, &pubkey_);
//----->>> BUG!!!!
    }
    pubkey_id_ = std::string(buf, n);
}
```

3.30 High-AggregableKeypair objects aggregated

SEVERITY:**HIGH****STATUS:****Fixed**

DESCRIPTION:

In `AggregableKeypair::AggregateKeypair(const std::vector<std::string>& keypair_vec)` the temporary vector of `AggregableKeypair` is built incorrectly:

```
Status AggregableKeypair::AggregateKeypair(const
std::vector<std::string>& keypair_vec) {
    std::vector<AggregableKeypair> in_vec;
    in_vec.resize(keypair_vec.size());

    for (const auto& keypair : keypair_vec) {
        AggregableKeypair in;
        auto ms = in.FromBytes(keypair);
        if (!ms.ok()) {
            return STACKED_STATUS(ErrorCode::CRYPTO_AGGREGATION_FAILED,
ms);
        }

        in_vec.emplace_back(std::move(in));
    }

    return AggregateKeypair(in_vec);
}
```

- `in_vec.resize(keypair_vec.size())` creates **N default-constructed** `AggregableKeypair` objects (with `pubkey_` never deserialized).
- The loop then `emplace_backs` N more initialized objects.
- Result: `in_vec.size() == 2N`; the first N entries are never initialized via `FromBytes`.
- `AggregateKeypair(const std::vector<AggregableKeypair>&)` later iterates over *all* entries and calls `blsPublicKeyAdd(&pubkey_, &keypair.pubkey_)`; on each, including these default-constructed ones.

This passes uninitialized/invalid BLS public keys into `blsPublicKeyAdd`, causing undefined behavior.

- `blsPublicKeyAdd` is invoked on `pubkey_` values that were never deserialized from valid serialized keys.

Refactor `AggregateKeypair` (`const std::vector<std::string>&`) so every element in `in_vec` is fully initialized before being passed to aggregation, and no extra default objects are left in the vector.

Use `reserve` and `emplace_back`:

```
// ...existing code...
Status AggregateKeypair::AggregateKeypair(const
    std::vector<std::string>& keypair_vec) {
    std::vector<AggregateKeypair> in_vec;
    in_vec.reserve(keypair_vec.size()); //-----> fix

    for (const auto& keypair : keypair_vec) {
        AggregateKeypair in;
        auto ms = in.FromBytes(keypair);
        if (!ms.ok()) {
            return STACKED_STATUS(ErrorCode::CRYPTO_AGGREGATION_FAILED,
ms);
        }

        in_vec.emplace_back(std::move(in));
    }

    return AggregateKeypair(in_vec);
}
// ...existing code...
```

3.31 High-Recursive Stack Overflow in CallTraceToJson

SEVERITY:**HIGH****STATUS:****Fixed****DESCRIPTION:**

The `CallTraceToJson` function is recursive. It processes a `CallTracer` object and then iterates through its sub-calls `call_tracer.calls`, calling itself for each one. There is no check for recursion depth. An attacker can craft a transaction with a deeply nested call structure that, when traced via an RPC call like `debug_traceTransaction`, will cause the `CallTraceToJson` function to recurse until it exhausts the stack, leading to a stack overflow and crashing the node.

```
// ...existing code...
    if (call_tracer.calls.Size() != 0) {
        Json::Value calls(Json::arrayValue);

        for (size_t i = 0; i < call_tracer.calls.Size(); i++)
            auto sub_call_data =
                EthJrpcConverter::CallTraceToJson(call_tracer.calls[i]);
        calls.append(sub_call_data);
    }

    data['calls'] = calls;
}
// ...existing code...
```

RECOMMENDATIONS:

Introduce a recursion depth limit. The function should accept a depth parameter.

3.32 High-Sensitive Key Material Not Cleared from Memory

SEVERITY:**HIGH****STATUS:****Fixed****DESCRIPTION:**

The private key is stored in a local variable `privkey` in the `GenerateKey` function but is never explicitly zeroed out or cleared from memory after use:

```
/* static */ Status Bls12_381SignerV1::GenerateKey(Keypair& keypair) {  
    // ...  
    blsSecretKey privkey;  
    // ...  
    {  
        RLockGuard gu(bls_init_lock_);  
        blsSecretKeySetByCSPRNG(&privkey);  
        // ... use the key ...  
    }  
    // Function returns without clearing privkey  
    return Status::OK;  
}
```

3.33 Medium-Race Condition in Task Counting

SEVERITY:**MEDIUM****STATUS:****Fixed**

DESCRIPTION:

The `EnqueueImpl` method in `thread_pool.h` contains a race condition that could allow the task count to exceed the specified maximum limit.

The Problematic Code Pattern

```
// First block with mutex protection
{
    std::unique_lock<std::mutex> lock(task_mutex_);

    if (is_always_wait) {
        task_cv_.wait(lock, [this]() {
            return task_count_.load() < max_task_count_.load();
        });
    } else if (task_count_.load() >= max_task_count_.load()) {
        // Log and return false
        return false;
    }
} // Lock is released here
TaskIncrement(timer); // Counter increment happens AFTER lock release
```

How the Race Condition Occurs

1. **Check Phase:** Thread A acquires the lock, checks that `task_count_ < max_task_count_`, then releases the lock
2. **Window of Vulnerability:** After Thread A releases the lock but before it calls `TaskIncrement()`
3. **Concurrent Access:** Thread B acquires the lock, sees that `task_count_` is still below `max_task_count_`, and also decides to proceed
4. **Limit Exceeded:** Both Thread A and Thread B (and potentially more threads) increment the counter, pushing it beyond the intended maximum

3.34 Medium-Unsafe Busy Wait in Destructor

SEVERITY:**MEDIUM****STATUS:****Fixed****DESCRIPTION:**

```
while (true) {  
    if (routine_finished_) {  
        break;  
    }  
    std::this_thread::sleep_for(std::chrono::milliseconds(10));  
}
```

Issue: This busy wait has no timeout, potentially causing the destructor to hang indefinitely if `routine_finished_` is never set to true.

3.35 Medium-Iterator Dereference After End Check

SEVERITY:**MEDIUM****STATUS:****Fixed**

DESCRIPTION:

The `InitializeImpl()` method in `aldaba-ng/services/controller/controller_service.cpp` contains a iterator dereference bug where `iter->second` is accessed outside the bounds check, potentially causing program crash when the configuration key is missing.

```
bool ControllerService::InitializeImpl() {  
  
    // ...  
  
    consensus_type_ = CONSENSUS_ALGORITHM_MYTUMBLER;  
  
    const std::map<std::string, std::string>* base_block_config =  
    init_consensus_config_ptr->epoch_config_.get();  
  
    auto iter = base_block_config->find(CONSENSUS_ALGORITHM_KEY);  
    if (iter != base_block_config->end()) {  
  
        consensus_type_ = iter->second; // Safe: within bounds check  
    }  
  
    LOGF_INFO('consensus type is {}', iter->second.c_str()); // Dangerous:  
        outside bounds check  
  
    // ...  
}
```

RECOMMENDATIONS:

Move the iterator dereference inside the bounds check or add proper error handling:

```
auto iter = base_block_config->find(CONSENSUS_ALGORITHM_KEY);
```

```
if (iter != base_block_config->end()) {  
consensus_type_ = iter->second;  
  
+   LOGF_INFO('consensus type is {}', iter->second.c_str()); // Safe:  
+   within bounds check  
  
+} else {  
  
+   LOGF_INFO('consensus type is {} (default)', consensus_type_); //  
+   Use default value  
  
}
```

3.36 Medium-Infinite Wait in ReadBlockSeqMap During Initialization

SEVERITY:**MEDIUM****STATUS:****Fixed****DESCRIPTION:**

The `ReadBlockSeqMap()` method in `aldaba-ng/services/controller/controller_service.cpp` contains an infinite wait loop without proper exit conditions, potentially causing service startup to hang indefinitely when data is missing or corrupted.

```
bool ControllerService::ReadBlockSeqMap(BlockNumber block_num,
                                         Seq& seq_num,
                                         bool& is_last_block_in_seq,
                                         bool& is_epoch_end) {

    uint32_t retry_interval_ms = 100;
    uint32_t retry_times = 0;
    while (true) { // infinite loop without exit conditions
        StorageHandleCode result =
chain_provider_->ReadBlockSeqMap(block_num,

seq_num,

is_last_block_in_seq,

is_epoch_end);

        if (result == StorageHandleCode::SUCCESS) {
            break; // Only exit on success
        } else {
            LOGF_WARN('read block: {} seq map not found, waiting for
write complete (attempt {})',
                    block_num,
                    retry_times);

            std::this_thread::sleep_for(std::chrono::milliseconds(retry_interval_ms));
            retry_times++; // Counter incremented but not used for exit
condition
        }
    }
}
```

```
    return true;
}
```

This method is called during service initialization in `HandleConsensusedResultAfterInit()` which is invoked from `StartupImpl()`, making it a critical path for service startup.

The `PushConsensusResult()` method demonstrates proper protection with `stopped_` flag checking:

```
void ControllerService::PushConsensusResult(...) {
    int ret = -1;
    while (ret && !stopped_) { // Proper: checks stopped_ flag
        // ...
        cd.Wait();
    }
}
```

RECOMMENDATIONS:

Add proper exit conditions to prevent infinite waiting:

```
bool ControllerService::ReadBlockSeqMap(BlockNumber block_num,
                                         Seq& seq_num,
                                         bool& is_last_block_in_seq,
                                         bool& is_epoch_end) {

    uint32_t retry_intervel_ms = 100;
    uint32_t retry_times = 0;
    const uint32_t max_retry_times = 1000; // Add retry limit
-   while (true) {
+   while (retry_times < max_retry_times && !Stopped()) { // Add proper
    exit conditions
        StorageHandleCode result =
        chain_provider_->ReadBlockSeqMap(block_num,

        seq_num,

        is_last_block_in_seq,

        is_epoch_end);

        if (result == StorageHandleCode::SUCCESS) {
            break;
        }
    }
}
```

```
    } else {
        LOGF_WARN('read block: {} seq map not found, waiting for
write complete (attempt {})',
                block_num,
                retry_times);

        std::this_thread::sleep_for(std::chrono::milliseconds(retry_interval_ms));
        retry_times++;
    }
}

+   if (retry_times >= max_retry_times) {
+       LOGF_ERROR('ReadBlockSeqMap timeout after {} attempts for block
+ {}', max_retry_times, block_num);
+       return false;
+   }

+   if (Stopped()) {
+       LOGF_INFO('ReadBlockSeqMap interrupted due to service shutdown
+ for block {}', block_num);
+       return false;
+   }

    return true;
}
```

3.37 Medium-Missing Capacity Check in ABuffer CutTail Method

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**`aldaba-ng/libraries/common/aldaba_buffer.h`**DESCRIPTION:**

The `CutTail()` method in `aldaba-ng/libraries/common/aldaba_buffer.h` lacks a capacity bounds check, potentially allowing buffer size to exceed the allocated memory capacity.

```
// aldaba-ng/libraries/common/aldaba_buffer.h
inline void CutTail(size_t sz) noexcept {
    assert(Left() <= sz);
    tail_ = head_ + sz;
}
```

RECOMMENDATIONS:

Add the missing capacity check:

```
inline void CutTail(size_t sz) noexcept {
    assert(Left() <= sz);
+   if (sz > capacity_) {
+       throw std::runtime_error('CutTail: size exceeds buffer capacity');
+   }
    tail_ = head_ + sz;
}
```

3.38 Medium-DOG Service Array Bounds Checking Leading to DoS

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**

aldaba-ng/services/dog/dog_service.cpp

DESCRIPTION:

`GetForwardTransactionRank()` and `ComputeForwardTarget()` methods perform `AtView(0)` operations without checking if nested lists are empty, leading to array bounds violations when processing malicious network packets.

```
// aldaba-ng/services/dog/dog_service.cpp

bool DOGService::GetForwardTransactionRank(...) {
    auto batches_view = view.ForwardBatchsView();
    if (batches_view.Size(ssz_version) == 0) {
        return false;
    }
    auto first_batch_view = batches_view.AtView(0, ssz_version);
    auto tx_list_view = first_batch_view.ForwardTxListView();
    auto tx_info = tx_list_view.AtView(0, ssz_version); // Bounds
violation
}

bool DOGService::ComputeForwardTarget(...) {
    auto batches_view = view.ForwardBatchsView();
    if (batches_view.Size(ssz_version) == 0) {
        return false;
    }
    auto first_batch_view = batches_view.AtView(0, ssz_version);
    auto tx_list_view = first_batch_view.ForwardTxListView();
    auto tx_info = tx_list_view.AtView(0, ssz_version); // Bounds
violation
}
```

RECOMMENDATIONS:

Add bounds checking for nested list access.

```
bool DOGService::GetForwardTransactionRank(...) {
    auto batches_view = view.ForwardBatchsView();
    if (batches_view.Size(ssz_version) == 0) {
        return false;
    }

    auto first_batch_view = batches_view.AtView(0, ssz_version);
    auto tx_list_view = first_batch_view.ForwardTxListView();

+   if (tx_list_view.Size(ssz_version) == 0) {
+       return false;
+   }

    auto tx_info = tx_list_view.AtView(0, ssz_version);
}

bool DOGService::ComputeForwardTarget(...) {
    auto batches_view = view.ForwardBatchsView();
    if (batches_view.Size(ssz_version) == 0) {
        return false;
    }

    auto first_batch_view = batches_view.AtView(0, ssz_version);
    auto tx_list_view = first_batch_view.ForwardTxListView();

+   if (tx_list_view.Size(ssz_version) == 0) {
+       return false;
+   }

    auto tx_info = tx_list_view.AtView(0, ssz_version);
}
```

3.39 Medium-DOG Service DNS Hijacking Leading to DoS/Eclipse Attack

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**

aldaba-ng/services/dog/p2p/fc_manager.cpp

DESCRIPTION:

`FcManager::OnCheckConn()` method performs DNS resolution without validation, allowing attackers to redirect connections to malicious servers through DNS hijacking. While cryptographic handshake prevents direct session compromise, this enables DoS and Eclipse attacks by isolating nodes from the network.

```
// aldaba-ng/services/dog/p2p/fc_manager.cpp

void FcManager::OnCheckConn() {
    // Get disconnected nodes
    std::vector<NodeInfo> disconnect_nodes;
    {
        std::lock_guard<std::mutex> guard(nodes_mutex_);
        node_manager_->GetDisconnectedNodes(disconnect_nodes);
    }

    for (auto node_info : disconnect_nodes) {
        // DNS resolution without validation
        boost::asio::io_service io_svc;
        boost::asio::ip::tcp::resolver r(io_svc);
        boost::system::error_code ec;
        auto it = r.resolve(domain_name, std::to_string(access_port), ec);
        if (it != it_end) { // use first
            dest_ep = *it; // Direct use of DNS result without validation
        }

        // Attempt connection to potentially malicious IP
        std::shared_ptr<Channel> channel =
            tcp_server_->CreateChannel(dest_ep,
            handler_thread_group_->GetIoContext());
        channel->Connect([this, self](const std::error_code& error, const
            std::shared_ptr<Channel>& chan) {
```

```
        if (error) {
            LOGF_ERROR('connect node failed '); // Connection fails
        } else {
            OnConnect(chan, NetworkType::TCP, true); // Handshake
will fail
        }
    });
}

// Retry mechanism continues the attack

conn_timer_ -> ExpiresAfter(std::chrono::milliseconds(option_.connection_check_interval));
conn_timer_ -> AsyncWait(std::bind(&FcManager::OnCheckConn, this));
}
```

SCENARIO:

1. Attacker performs DNS poisoning/hijacking
2. Node A queries domain for Node B -> DNS returns attacker's IP
3. Node A connects to attacker's server -> Handshake fails -> Connection drops
4. Retry mechanism attempts reconnection -> Same malicious IP -> Repeat
5. Node A becomes isolated, unable to communicate with legitimate peers

RECOMMENDATIONS:

- When an IP address resolved via DNS fails during the subsequent cryptographic handshake, the system shall clearly log this as an “address verification failure” event rather than an ordinary “network timeout”.
- The system should then associate this resolved malicious IP address with its corresponding domain name, add it to a temporary blacklist, and refrain from attempting to connect to this IP for a subsequent period (e.g., 10 minutes).

3.40 Medium-DOG Service Unauthenticated Node Addition Leading to Resource Exhaustion

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**

aldaba-ng/services/dog/p2p/fc_manager.cpp

DESCRIPTION:

`FcManager::IsConnectable()` method automatically adds unknown node IDs to the trusted node list before cryptographic handshake verification, allowing remote attackers to pollute the node list and exhaust memory and CPU resources through a simple TCP connection attack.

```
// aldaba-ng/services/dog/p2p/fc_manager.cpp

bool FcManager::IsConnectable(NodeId& node_id, bool incoming) {
    NodeInfo node_info;
    {
        std::lock_guard<std::mutex> guard(nodes_mutex_);
        if (auto node = node_manager_->FindNodeById(node_id)) {
            node_info = node->GetNodeInfo();
        } else {
            LOGF_WARN("The node {} is not found in aldaba!",
toHex(node_id).c_str());
            node_info.node_id_ = node_id;
            node_info.node_role_ = NodeRole::NODE_ROLE_NONCONSENSUS;
            node_info.node_state_ = ContractNodeState::NODE_STATE_NORMAL;
            node_manager_->AddNode(node_info); // adds unauthenticated
nodes
        }
    }
    // ... connection limit checks
    return true;
}

void NodeManager::AddNode(const NodeInfo& info) {
    // delete node if it already exist
    if (node_set_.count(info.node_id_)) {
        auto node = node_set_[info.node_id_];
```

```
        node->Disconnect();
    }

    LOGF_INFO('AddNode {}', toHex(info.node_id_));
    node_set_[info.node_id_] = std::make_shared<Node>(info, host_,
conn_timeout_);
}

void NodeManager::GetDisconnectedNodes(std::vector<NodeInfo>& nodes) {
    for (auto node : node_set_) { // O(n) traversal of all nodes
        LOGF_DEBUG('check nodes, id: {}, connected: {}, disable: {}',
            NODE_STR(node.first),
            node.second->IsConnected(),
            node.second->IsDisabled());
        if (!node.second->IsConnected() && !node.second->IsDisabled()) {
            nodes.push_back(node.second->GetNodeInfo());
        }
    }
}
```

SCENARIO:

1. Attacker establishes TCP connection to target node
2. Attacker sends handshake packet with random/fake node_id
3. Server calls `IsConnectable()` before handshake verification
4. Unknown node_id triggers automatic addition to node list
5. Handshake fails, connection drops, but node remains in list
6. Attacker repeats with different node_id every millisecond
7. Node list grows exponentially, causing resource exhaustion

RECOMMENDATIONS:

`FcManager::IsConnectable()` remove the logic that automatically adds unknown nodes, and instead directly reject connection requests from unknown nodes.

`FcManager::OnHandShake()` new nodes should be added to the node manager only after the handshake verification is successful.

`FcManager::OnError()` when handshake verification fails, unauthenticated nodes should be checked and deleted to prevent the node list from being contaminated by malicious nodes.

3.41 Medium-DOG Service Unbounded Pending Connections Leading to Resource Exhaustion

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**

aldaba-ng/services/dog/p2p/fc_manager.cpp

DESCRIPTION:

`FcManager::OnCheckConn()` method attempts to establish connections to all disconnected nodes simultaneously without resource limits, creating a **resource amplification** that can exhaust file descriptors, memory, and CPU when combined with node list pollution attacks.

```
// aldaba-ng/services/dog/p2p/fc_manager.cpp

void FcManager::OnCheckConn() {
    // Get all disconnected nodes
    std::vector<NodeInfo> disconnect_nodes;
    {
        std::lock_guard<std::mutex> guard(nodes_mutex_);
        node_manager_->GetDisconnectedNodes(disconnect_nodes);
    }

    LOGF_DEBUG('disconnect_nodes: {}', disconnect_nodes.size());

    // Attempt to connect to ALL disconnected nodes simultaneously
    for (auto node_info : disconnect_nodes) { // No limit on iteration
        count
        auto node_id = node_info.node_id_;
        if (!IsConnectable(node_id, false)) {
            continue;
        }

        // Create connection channel for each node
        std::shared_ptr<Channel> channel =
        tcp_server_->CreateChannel(dest_ep,
        handler_thread_group_->GetIoContext());
        {
            std::lock_guard<std::mutex> lock(pending_chs_mutex_);
```

```

        pending_channels_.insert(channel); // Add to pending set
    }

    // Initiate connection
    auto self = shared_from_this();
    channel->Connect([this, self](const std::error_code& error, const
std::shared_ptr<Channel>& chan) {
        if (error) {
            LOGF_ERROR('connect node failed, channelId={}, error code
{}, msg: {}',
                        chan->GetId(), error.value(),
error.message().c_str());
        } else {
            OnConnect(chan, NetworkType::TCP, true);
        }
    });
}

// Reset timer for next connection check

conn_timer_->ExpiresAfter(std::chrono::milliseconds(option_.connection_check_interval_));
conn_timer_->AsyncWait(std::bind(&FcManager::OnCheckConn, this));
}

void NodeManager::GetDisconnectedNodes(std::vector<NodeInfo>& nodes) {
    for (auto node : node_set_) { // 0(n) traversal of all nodes
        LOGF_DEBUG('check nodes, id: {}, connected: {}, disable: {}',
                    NODE_STR(node.first),
                    node.second->IsConnected(),
                    node.second->IsDisabled());
        if (!node.second->IsConnected() && !node.second->IsDisabled()) {
            nodes.push_back(node.second->GetNodeInfo()); // Add to
connection attempt list
        }
    }
}
}

```

SCENARIO:

1. Attacker has polluted node list with 50,000 fake nodes
2. `conn_timer_` automatically triggers `OnCheckConn()` every `connection_check_interval_`

milliseconds

3.

- `GetDisconnectedNodes()` returns 50,000 fake nodes
- Loop creates 50,000 `Channel` objects simultaneously
- 50,000 `channel->Connect()` calls are made
- Operating system receives 50,000 outbound connection requests

RECOMMENDATIONS:

Implement connection rate limiting and resource protection.

3.42 Medium-Rate Limiting Logic Flaw in ETHSendRawTransaction Allows DoS Bypass

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**

aldaba-ng/services/portal/dispatcher.cpp

DESCRIPTION:

`ETHSendRawTransaction()` rate limiting check only returns error response but continues processing the transaction, bypassing the intended protection mechanism.

```
// aldaba-ng/services/portal/dispatcher.cpp
void Dispatcher::ETHSendRawTransaction(std::shared_ptr<lightnet::Context>
    ctx) {
    uint32_t load = health_checker->GetLoad();
    if (load >= config_.max_dispatch_load_limit_) {
        CETINA_COUNTER_INC(TX_EXCEED_THROTTLE_COUNT);
        ServeError(ctx, JSONRPCError::RATE_LIMITED, 'THROTTLED: txpool is
        overload');
    }

    auto tx_ptr = std::make_shared<Transaction>();
    // ...

    eth_worker_group->PushPartitionTx(partition_id,
    std::move(tx_dispatch));
}
```

IMPACT:

Attackers can completely bypass rate limiting by ignoring error responses and continuing to send requests, as each request triggers full processing including transaction parsing, validation, and backend queue insertion, leading to resource exhaustion and DoS.

RECOMMENDATIONS:

Add `return;` statement after `ServeError()` call.

3.43 Medium-Rate Limiter Initialized But Never Used

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**`aldaba-ng/services/portal/dispatcher.cpp`**DESCRIPTION:**

The `rate_limiter_` in Portal service is correctly initialized in `Initialize()` but never called throughout the service, causing rate limiting functionality to be completely ineffective.

```
// aldaba-ng/services/portal/dispatcher.cpp
bool Dispatcher::Initialize(const DispatcherConfig& config) {
    config_ = config;
    rate_limiter_.reset(
        new RateLimiter(config.max_dispatch_tx_rate_,
            config.max_dispatch_tx_burst_));
    return true;
}
```

RECOMMENDATIONS:

Add rate limiting check before processing requests.

3.44 Medium-ETH Call Pending State Implementation Violates Ethereum JSON-RPC Specification

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**

aldaba-ng/services/portal/dispatcher.cpp

DESCRIPTION:

The `eth_call` implementation in portal service incorrectly handles the `'pending'` block parameter by treating it identically to `'latest'`, violating the Ethereum JSON-RPC API specification. And only support query the latest blocknumber.

```
// aldaba-ng/services/portal/dispatcher.cpp
void Dispatcher::ETHCall(std::shared_ptr<lightnet::Context> ctx) {
    // ...
    if (!params[1].isString()) {
        block_number = LATEST_BLOCK_NUMBER;
    } else {
        auto bn_str = params[1].asString();
        if (bn_str == 'earliest') {
            block_number = LATEST_BLOCK_NUMBER;
        } else if (bn_str == 'latest') {
            block_number = LATEST_BLOCK_NUMBER;
        } else if (bn_str == 'pending') {
            block_number = LATEST_BLOCK_NUMBER;
        } else if (bn_str == 'finalized') {
            block_number = LATEST_BLOCK_NUMBER;
        } else {
            PARSE_HEX_DECIMAL_UINT64(block_number, bn_str, 'block
number');
            if (block_number > stabl_block_number) {
                return {JSONRPCError::INVALID_PARAMS,
                    'PARAM_VERIFY_ERROR: block number larger than
latest'};
            }
        }
    }
}
```

image.png

https://ethereum.org/en/developers/docs/apis/json-rpc/#eth_call

RECOMMENDATIONS:

If the functionality is not yet complete, return an appropriate error instead of providing incorrect information to users.

```
} else if (bn_str == 'earliest') {  
-   block_number = LATEST_BLOCK_NUMBER;  
+   return {JSONRPCError::INVALID_PARAMS, 'earliest block not supported  
    yet'};  
} else if (bn_str == 'latest') {  
    block_number = LATEST_BLOCK_NUMBER;  
} else if (bn_str == 'pending') {  
-   block_number = LATEST_BLOCK_NUMBER;  
+   return {JSONRPCError::INVALID_PARAMS, 'pending state not supported  
    yet'};  
} else if (bn_str == 'finalized') {  
-   block_number = LATEST_BLOCK_NUMBER;  
+   return {JSONRPCError::INVALID_PARAMS, 'finalized block not supported  
    yet'};
```

3.45 Medium-Unsafe File Permissions Due to `umask(0)` in Daemon Mode

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**

aldaba-ng/libraries/app/app.cpp

DESCRIPTION:

daemon setup process sets `umask(0)`, creating world-writable files that can be exploited by local users.

```
// aldaba-ng/libraries/app/app.cpp
bool App::SetupDaemon() {
    // ...
    setsid();
    umask(0);

    if (!RedirectIO()) {
        LOGF_INFO('redirect error');
        return false;
    }
}

bool App::RedirectIO() {
    FILE* fp1 = freopen('./stdout.log', 'w', stdout); // 0666 permissions
    FILE* fp2 = freopen('./stderr.log', 'w', stderr); // 0666 permissions
}

void App::GeneratePidFile() {
    std::ofstream of((JoinPath(cwd_, pid_file_)).c_str(),
                     std::ofstream::out | std::ofstream::trunc); // 0666
    permissions
}
```

With `umask(0)`, files are created with 0666 permissions (world writable), allowing local users to tamper with logs, fill disk space and inject malicious data.

RECOMMENDATIONS:

Use restrictive umask and explicit file permissions.

```
bool App::SetupDaemon() {  
    // ...  
    setsid();  
-   umask(0);  
+   umask(027); // Files: 0640, Directories: 0750  
}
```

3.46 Medium-Component Shutdown Failures Ignored in ShutdownComponents

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**

aldaba-ng/libraries/app/app.cpp

DESCRIPTION:

`ShutdownComponents()` ignores return values from component `Shutdown()` calls, silently masking shutdown failures that could lead to resource leaks and incomplete cleanup.

```
// aldaba-ng/libraries/app/app.cpp
void App::ShutdownComponents() {
    for (auto it = component_list_.rbegin(); it !=
         component_list_.rend(); ++it) {
        if (*it == 'aldaba::Storage') {
            auto storage_component = GET_COMPONENT(Storage);
            if (storage_component) {
                storage_component->Shutdown(); // Return ignored
                storage_component->Destroy();
            }
        } else if (*it == 'aldaba::StateProviderFactory') {
            auto state_factory =
                GET_COMPONENT(aldaba::StateProviderFactory);
            if (state_factory) {
                state_factory->Shutdown(); // Return ignored
            }
        }
        // ...
    }
}
```

RECOMMENDATIONS:

Check and handle component shutdown return values.

```
void App::ShutdownComponents() {
+   bool all_success = true;
```



```
    for (auto it = component_list_.rbegin(); it !=
        component_list_.rend(); ++it) {
        if (*it == 'aldaba::Storage') {
            auto storage_component = GET_COMPONENT(Storage);
            if (storage_component) {
-                storage_component->Shutdown();
+                if (!storage_component->Shutdown()) {
+                    printf('Failed to shutdown Storage component\n');
+                    all_success = false;
+                }
                storage_component->Destroy();
            }
        }
+        // ... handle other components similarly
    }
+    if (!all_success) {
+        printf('Some components failed to shutdown properly\n');
+    }
}
```

3.47 Medium-CreateSessionForHttpServer Bypasses Session Limit

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**

aldaba-ng/libraries/lightnet/session_manager.cpp

DESCRIPTION:

`CreateSessionForHttpServer()` lacks the session count limit check that exists in `CreateSession()`, allowing unlimited HTTP session creation and bypassing the `max_sessions_` resource limit.

```
// aldaba-ng/libraries/lightnet/session_manager.cpp
std::shared_ptr<Session> SessionManager::CreateSession(ProtocolType
    protocol_type,

    std::shared_ptr<SSLCtx> ssl_ctx,

                                     bool is_client) {
    if (GetSessionsSize() >= max_sessions_) { //limit check
        LOGF_ERROR('session size is overloaded, max is {}',
max_sessions_);
        return nullptr;
    }
    // ...
}

std::shared_ptr<Session> SessionManager::CreateSessionForHttpServer() {
    // bypasses max_sessions_
    std::shared_ptr<Session> session =
std::make_shared<HttpSession>(SessionType::SERVER,

    ProtocolType::HTTP,

    shared_from_this(),

    GetIoContext(),

    kMaxBufferSize,
```

```
    max_batch_request_size_,  
  
    http_session_idle_timeout_,  
  
    enable_http_long_connection_);  
    HandleSessionCreate(session);  
    return session;  
}
```

RECOMMENDATIONS:

Add the same session limit check to `CreateSessionForHttpServer()`.

```
// aldaba-ng/libraries/lightnet/session_manager.cpp  
std::shared_ptr<Session> SessionManager::CreateSessionForHttpServer() {  
+   if (GetSessionsSize() >= max_sessions_) {  
+       LOGF_ERROR('session size is overloaded, max is {}',  
+         max_sessions_);  
+       return nullptr;  
+   }  
+  
+ }
```

3.48 Medium-Null Pointer Dereference in EVMTxGenerator::SetEvmType

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**

aldaba-ng/libraries/utils/evm_tx_generator.cpp

DESCRIPTION:

`EVMTxGenerator::SetEvmType` fails to initialize `helper_` for unsupported `eth_type` values but continues execution.

```
// aldaba-ng/libraries/utils/evm_tx_generator.cpp
void EVMTxGenerator::SetEvmType(std::string eth_type,
                                Json::Value eth_conf,
                                uint32_t connections_size,
                                bool load_imbalance) {

    // initialize helper
    if (eth_type == 'transfer') {
        helper_ = std::make_shared<EVMTTransfer>(connections_size,
load_imbalance);
    } else if (eth_type == 'erc20') {
        helper_ = std::make_shared<EVMerc20>(connections_size,
load_imbalance);
    } else if (eth_type == 'erc721') {
        helper_ = std::make_shared<EVMerc721>(connections_size,
load_imbalance);
    } else if (eth_type == 'rwa') {
        helper_ = std::make_shared<EVMRwa>(connections_size,
load_imbalance);
    } else if (eth_type == 'json_api') {
        helper_ = std::make_shared<EVMJsonAPI>(connections_size,
load_imbalance);
    } else {
        LOGF_ERROR('not support eth_type {}', eth_type);
    }

    helper_>SetConfig(eth_conf);
    eth_type_ = eth_type;
}
```

RECOMMENDATIONS:

```
void EVMTxGenerator::SetEvmType(std::string eth_type,
                                Json::Value eth_conf,
                                uint32_t connections_size,
                                bool load_imbalance) {
    // initialize helper
    if (eth_type == 'transfer') {
        helper_ = std::make_shared<EVMTTransfer>(connections_size,
load_imbalance);
    } else if (eth_type == 'erc20') {
        helper_ = std::make_shared<EVMErc20>(connections_size,
load_imbalance);
    } else if (eth_type == 'erc721') {
        helper_ = std::make_shared<EVMerc721>(connections_size,
load_imbalance);
    } else if (eth_type == 'rwa') {
        helper_ = std::make_shared<EVMRwa>(connections_size,
load_imbalance);
    } else if (eth_type == 'json_api') {
        helper_ = std::make_shared<EVMJsonAPI>(connections_size,
load_imbalance);
    } else {
        LOGF_ERROR('not support eth_type {}', eth_type);
+       return; // Early return to prevent null pointer dereference
    }

    helper_>SetConfig(eth_conf);
    eth_type_ = eth_type;
}
```

3.49 Medium-Hardcoded Timezone Offset in Time Utility Functions

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**

aldaba-ng/libraries/utils/time.cpp

DESCRIPTION:

Hardcodes an UTC+8 in multiple time functions, assuming the system always runs in China Standard Time. but this creates incorrect timestamps when deployed in other timezones.

```
// aldaba-ng/libraries/utils/time.cpp
static int64_t CurrentTimeInSecond() {
    TimeType epoch(boost::gregorian::date(1970, 1, 1));
    return (CurrentTime() - epoch).total_seconds() - (8 * 3600);
}

static int64_t CurrentTimeInMillisecond() {
    TimeType epoch(boost::gregorian::date(1970, 1, 1));
    return (CurrentTime() - epoch).total_milliseconds() - (8 * 3600 * 1000);
}

static int64_t CurrentTimeInMicroseconds() {
    TimeType epoch(boost::gregorian::date(1970, 1, 1));
    return (CurrentTime() - epoch).total_microseconds() - int64_t(8LL * 3600 * 1000 * 1000);
}

static int64_t CurrentTimeInNanoseconds() {
    TimeType epoch(boost::gregorian::date(1970, 1, 1));
    return (CurrentTime() - epoch).total_nanoseconds()
        - int64_t(8LL * 3600 * 1000 * 1000 * 1000);
}
```

IMPACT:

Incorrect timestamps in non-UTC+8 timezones (e.g., 13-hour difference in EST, 9-hour difference in CET)

RECOMMENDATIONS:

use UTC time directly instead of hardcoded offset or local time.

3.50 Medium-Buffer Overflow in BitMapper::GenerateBitmap

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**

aldaba-ng/libraries/crypto/impl/mycrypto.cpp

DESCRIPTION:

an off by one boundary check that allows buffer overflow. allows `index == size` to pass validation, heap buffer overflow when setting bits.

```
// aldaba-ng/libraries/crypto/impl/mycrypto.cpp
/* static */ BitMap BitMapper::GenerateBitmap(const std::vector<size_t>&
indexes, size_t size) {
    auto result = GenerateBitmap(size);

    for (auto index : indexes) {
        if (index > size) {
            throw std::out_of_range('BitMap index bigger than size');
        }
        BitMapInnerSet(result, index, true); //
    }

    return result;
}
```

```
// If size = 8, GenerateBitmap(8) allocates:
// - MYCRYPTO_ID_SIZE (2 bytes) + REMAINDER_SIZE (1 byte) + 1 data byte =
//   4 bytes total
// - Valid data indices: 0, 1, 2, 3
// - Size(result) returns 8 (number of bits)

// When index = 8:
// - Check: (8 > 8) = false, passes validation
// - BitMapInnerSet: i = 8/8 = 1, accesses bitmap[1 + 2 + 1] = bitmap[4]
// - But result only has 4 bytes (indices 0-3), so bitmap[4] is heap
//   overflow
```


RECOMMENDATIONS:

```
/* static */ BitMap BitMapper::GenerateBitmap(const std::vector<size_t>&
indexes, size_t size) {
    auto result = GenerateBitmap(size);

    for (auto index : indexes) {
-       if (index > size) {
+       if (index >= size) {
            throw std::out_of_range('BitMap index bigger than size');
        }
        BitMapInnerSet(result, index, true);
    }

    return result;
}
```

3.51 Medium-AES-GCM IV Length Parameter Inconsistency

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**

aldaba-ng/libraries/crypto/impl/openssl_helpers.cpp

DESCRIPTION:

AES-GCM encryption and decryption functions have inconsistent IV length handling. When `iv.size() != 12`, the encryption function incorrectly sets the IV length to the default 12 bytes instead of the actual IV size, while the decryption function correctly uses the actual IV size.

```
// aldaba-ng/libraries/crypto/impl/openssl_helpers.cpp
// Encryption side - always sets IVLEN to 12 when iv.size() != 12
if (iv.size() != DEFAULT_AES_IV_LEN
    && 1
    != EVP_CIPHER_CTX_ctrl(evp_cipher_ctx.get(),
                           EVP_CTRL_AEAD_SET_IVLEN,
                           static_cast<int>(DEFAULT_AES_IV_LEN),
                           nullptr)) {
    return STATUS(ErrorCode::CRYPTO_PROVIDER_ERROR,
                  'failed @ set IVlen, openssl EVP_CIPHER_CTX_ctrl');
}
```

```
// Decryption side - uses actual iv.size()
if (iv.size() != DEFAULT_AES_IV_LEN
    && 1
    != EVP_CIPHER_CTX_ctrl(evp_cipher_ctx.get(),
                           EVP_CTRL_AEAD_SET_IVLEN,
                           static_cast<int>(iv.size()),
                           nullptr)) {
    return STATUS(ErrorCode::CRYPTO_PROVIDER_ERROR, 'failed @ openssl
    EVP_CIPHER_CTX_ctrl');
}
```

RECOMMENDATIONS:

Fix encryption side to match decryption side behavior.

```
if (iv.size() != DEFAULT_AES_IV_LEN
    && 1
        != EVP_CIPHER_CTX_ctrl(evp_cipher_ctx.get(),
                                EVP_CTRL_AEAD_SET_IVLEN,
-                                static_cast<int>(DEFAULT_AES_IV_LEN),
+                                static_cast<int>(iv.size()),
                                nullptr)) {
    return STATUS(ErrorCode::CRYPTO_PROVIDER_ERROR,
                  'failed @ set IVlen, openssl EVP_CIPHER_CTX_ctrl');
}
```

3.52 Medium-AES-GCM Random Key Generation Only Assigns First Byte

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**

aldaba-ng/libraries/crypto/impl/local/openssl_aes_gcm_cipher_v1.cpp

DESCRIPTION:

When generating a random key for AES-GCM encryption, creates 16 bytes of random data but only assigns the first byte to the key string. an invalid key length and severely reduced entropy.

```
// aldaba-ng/libraries/crypto/impl/local/openssl_aes_gcm_cipher_v1.cpp
Status OsslAesGcmCipherV1::SetKey(std::string& key) {
    if (key.empty()) {
        std::vector<unsigned char> rand_bytes(kDefaultKeyLen); // 16
        bytes
        RAND_bytes(rand_bytes.data(), static_cast<int>(kDefaultKeyLen));
        key.resize(kDefaultKeyLen); // Resize to 16 bytes
        key = *(reinterpret_cast<char*>(rand_bytes.data())); // Only
        assigns first byte
    }
    // ...
}
```

RECOMMENDATIONS:

```
Status OsslAesGcmCipherV1::SetKey(std::string& key) {
    if (key.empty()) {
        std::vector<unsigned char> rand_bytes(kDefaultKeyLen);
        RAND_bytes(rand_bytes.data(), static_cast<int>(kDefaultKeyLen));
-         key.resize(kDefaultKeyLen);
-         key = *(reinterpret_cast<char*>(rand_bytes.data()));
+         key.assign(reinterpret_cast<char*>(rand_bytes.data()),
+ kDefaultKeyLen);
    }
    // ...
}
```

3.53 Medium-RSA-OAEP Encryption and Decryption Use Incompatible Encoding

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**`aldaba-ng/libraries/crypto/impl/local/openssl_rsa_pkey_cipher_v1.cpp`**DESCRIPTION:**

RSA-OAEP encryption stores ciphertext as decimal text representation, but decryption expects binary data. encoding mismatch makes the encrypt/decrypt operations non invertible.

```
// aldaba-ng/libraries/crypto/impl/local/openssl_rsa_pkey_cipher_v1.cpp
// Encryption side - stores as decimal text
bytes raw_ciphertext(pkey_len_in);
// ...
PkeyCiphertext pkcipher_v1(algo, pubkey_encoded,
    BytesToString(raw_ciphertext));

// Decryption side - expects binary data
std::string ciphertext_str = pkcipher_v1.ciphertext();
bytes ciphertext_bytes = StringToBytes(ciphertext_str);
// ...
```

RECOMMENDATIONS:

Use binary encoding consistently for both encryption and decryption.

```
- PkeyCiphertext pkcipher_v1(algo, pubkey_encoded,
    BytesToString(raw_ciphertext));
+ PkeyCiphertext pkcipher_v1(algo, pubkey_encoded,
    std::string(reinterpret_cast<char*>(raw_ciphertext.data()),
    raw_ciphertext.size()));

- std::string ciphertext_str = pkcipher_v1.ciphertext();
- bytes ciphertext_bytes = StringToBytes(ciphertext_str);
+ std::string ciphertext_str = pkcipher_v1.ciphertext();
+ bytes ciphertext_bytes(ciphertext_str.begin(), ciphertext_str.end());
```

3.54 Low- Unused Buffer with Potential Memory Disclosure

SEVERITY:

LOW

STATUS:

Fixed

DESCRIPTION:

An unused buffer allocation in the Noise protocol handshake implementation could potentially lead to information disclosure through uninitialized memory content. The `Handshake::Run` method allocates a buffer that is never used but remains in memory:

```
outcome::result<void> Handshake::Run(const std::array<uint8_t,
    aldaba::NODE_ID_LENGTH>& identity) {
    // ... code ...
    const size_t dh25519_len = 32;
    const size_t poly1305_tag_size = 16;
    const size_t length_prefix_size = 2;
    size_t max_msg_size = 2 * dh25519_len + payload.size() + 2 *
    poly1305_tag_size;
    std::vector<uint8_t> buffer(max_msg_size + length_prefix_size); //
    Line 298 - UNUSED BUFFER

    if (initiator_) {
        // ... rest of function, buffer is never referenced ...
    } else {
        // ... rest of function, buffer is never referenced ...
    }
}
```

3.55 Low-Super Permission Design Flaw in InferRuleManager

SEVERITY:

LOW

STATUS:

Fixed

DESCRIPTION:

In `algebra-ng/sys_contract/artifact_src/solidity/rule_mng.sol` and `permission_control.sol`, the `checkSuperPermission()` function grants super privileges to `address(0)`, creating a potential security trap. While the current function calls pass `msg.sender` and cannot exploit this flaw, the design is inherently dangerous and could be exploited if the function signature changes in future implementations.

```
function checkSuperPermission(address _addr) private view
returns(bool) {
    if (_addr == administrator_ || _addr == address(0)) {
        return true;
    }

    return false;
}
```

This function is used in critical operations like `advanceEpoch()` and `transferSuperAdmin()`:

```
function advanceEpoch() external {
    require(checkSuperPermission(msg.sender), 'Permission denied');
    // ...
}

function transferSuperAdmin(address _new_admin) external {
    require(checkSuperPermission(msg.sender), 'Permission denied');
    // ...
}
```

IMPACT:

The design flaw grants super privileges to `address (0)`, creating a potential security trap that could be exploited if function signatures change in future implementations.

3.56 Low-keys repeated coverage

SEVERITY:

LOW

STATUS:

Fixed

DESCRIPTION:

In `aldaba-ng/sys_contract/artifact_src/solidity/sys_chaincfg.sol`, keys are not deduplicated, which may cause the same keys in `Config` to exist.

```
// sys_chaincfg.sol
function set_config(string[] memory keys, string[] memory values)
    external onlyOwner {      // @audit:
    .....
    if (old_config_cps_number != 0) {
        require(configCps[configCps.length - 2].effectiveBlockNum <=
            block.number, 'INVALID_STATE');

        for (uint256 i = 0; i < configCps[configCps.length -
            2].configs.length; i++) {
            Config storage conf = configCps[configCps.length -
            2].configs[i];
            bool found = false;
            for (uint256 j = 0; j < keys.length; j++) {
                if (keccak256(abi.encodePacked(conf.key)) ==
                    keccak256(abi.encodePacked(keys[j]))) {
                    found = true;
                    break;
                }
            }
            if (!found) {
                cfgCp.configs.push(Config({
                    key: conf.key,
                    value: conf.value
                }));
            }
        }
    }

    for (uint256 i = 0; i < keys.length; i++) {
        cfgCp.configs.push(Config({
            key: keys[i],
```

```
        value: values[i]
    }));
}

emit ConfigUpdate(uint64(block.number), uint64(block.number + 1),
keys, values);
}
```

`get_config` may return unexpected results when executed.

```
// sys_chaincfg.sol
function get_config(string memory key) public view returns (string
memory) {
    if (configCps.length != 0) {
        for (uint256 i = 0; i < configCps[configCps.length -
1].configs.length; i++) {
            Config storage conf = configCps[configCps.length -
1].configs[i];
            if (keccak256(abi.encodePacked(key)) ==
keccak256(abi.encodePacked(conf.key))) {
                return conf.value;
            }
        }
    }
    return '';
}
```

IMPACT:

When calling the `get_config` function, if the Config has the same key, an unexpected value may be returned.

3.57 Low-Out-of-Bounds Access Vulnerability in OnSysStakingEvent

SEVERITY:

LOW

STATUS:

Fixed

DESCRIPTION:

In `alibaba-ng/services/controller/controller_service.cpp`, the `OnSysStakingEvent()` function contains an out-of-bounds access vulnerability when processing contract events. The function checks `topics.Size() > 0` but directly accesses `topics[1]` without ensuring the array contains at least 2 elements.

```
void ControllerService::OnSysStakingEvent(const ContractEvent&
    sys_staking_event) {
    // ...
    for (const auto& event : sys_staking_event.events) {
        ssz::SSZObjectView<schema::Log> event_view(

ssz::Buffer{reinterpret_cast<uint8_t*>(const_cast<char*>(event.data()))},
event.size());
        auto topics = event_view.Topics();
        if (topics.Size() > 0
            && toHex(FixedListToString(topics[0])) ==
SYS_STAKING_CONTRACT_DOMAIN_UPDATE_EVENT) {
            DomainUpdateEvent domain_update_event{};
            std::string domain_id = toHex(FixedListToString(topics[1]));
            // second is poolid
            std::string data = VariableListToString(event_view.Data());
            domain_update_event.domain_id = domain_id;
            domain_update_event.ParseLog(data);
            LOGF_INFO('DomainUpdateEvent is {}',
domain_update_event.ToJsonString());

            OnDomainUpdate(domain_update_event);
        }
    }
}
```

The vulnerability occurs because:

1. The condition `topics.Size() > 0` only ensures the array has at least 1 element
2. The code directly accesses `topics[1]` (second element) without checking if it exists

3. If `topics.Size() == 1`, accessing `topics[1]` results in out-of-bounds memory access

RECOMMENDATIONS:

Fix the boundary check to ensure at least 2 topics exist before accessing `topics[1]`:

```
- if (topics.Size() > 0
+ if (topics.Size() >= 2
    && toHex(FixedListToString(topics[0])) ==
    SYS_STAKING_CONTRACT_DOMAIN_UPDATE_EVENT) {
    std::string domain_id = toHex(FixedListToString(topics[1]));
}
```

3.58 Low-SSZ Data Integrity Validation Bypass

SEVERITY:

LOW

STATUS:

Fixed

DESCRIPTION:

The `OnSysStakingEvent()` and `OnSysCfgmngEvent()` methods in `aldaba-ng/services/controller` directly access SSZ-encoded contract event data without performing integrity validation. The code constructs `ssz::SSZObjectView` objects and immediately accesses their fields without calling the `Verify()` method to ensure data structure integrity. Other components properly validate SSZ data using `SSZUnPackWithCode()` or explicit `Verify()` calls before accessing fields.

```
void ControllerService::OnSysStakingEvent(const ContractEvent&
    sys_staking_event) {
    // ...
    for (const auto& event : sys_staking_event.events) {
        ssz::SSZObjectView<schema::Log> event_view(

ssz::Buffer{reinterpret_cast<uint8_t*>(const_cast<char*>(event.data()))},
event.size());
        auto topics = event_view.Topics();
        if (topics.Size() > 0
            && toHex(FixedListToString(topics[0])) ==
SYS_STAKING_CONTRACT_DOMAIN_UPDATE_EVENT) {
            DomainUpdateEvent domain_update_event{};
            std::string domain_id = toHex(FixedListToString(topics[1]));
            // second is poolid
            std::string data = VariableListToString(event_view.Data());
            domain_update_event.domain_id = domain_id;
            domain_update_event.ParseLog(data);
            LOGF_INFO('DomainUpdateEvent is {}',
domain_update_event.ToJsonString());

            OnDomainUpdate(domain_update_event);
        }
    }
    // ...
}

void ControllerService::OnSysCfgmngEvent(const ContractEvent&
    sys_cfgmng_event) {
```

```
// ...
for (const auto& event : sys_cfgmng_event.events) {
    ssz::SSZObjectView<schema::Event> event_view(

ssz::Buffer{reinterpret_cast<uint8_t*>(const_cast<char*>(event.data()))},
event.size());
    auto topics = event_view.Topics();
    if (topics.Size() > 0 &&
VariableListToString(topics[0]).substr(1) == CONFIG_UPDATE_TOPIC) {
        auto desc = VariableListToString(event_view.Desc());
        ConfigUpdateEvent config_update_event;
        if (config_update_event.DataStreamDecode(desc)) {
            OnConfigUpdate(config_update_event);
        }
    }
}
// ...
}
```

RECOMMENDATIONS:

Add SSZ verification before accessing data to ensure data integrity:

```
ssz::SSZObjectView<schema::Log> event_view(

    ssz::Buffer{reinterpret_cast<uint8_t*>(const_cast<char*>(event.data()))},
    event.size());
+ if (event_view.Verify(0) != ssz::SSZError::OK) {
+     LOGF_WARN('Invalid SSZ event data, skipping');
+     continue;
+ }
auto topics = event_view.Topics();
```

3.59 Low-Invalid BLS Public Key Processing Continuation

SEVERITY:

LOW

STATUS:

Fixed

DESCRIPTION:

The `OnDomainUpdate()` method in `aldaba-ng/services/controller/controller_service.cpp` continues processing domain update events even when BLS aggregate public key parsing fails. The code calls `stabilizing_pubkey.FromBytes(event.stabilizing_pubkey)` and only logs an error when the parsing fails, but proceeds to update the validator set with potentially invalid cryptographic material.

```
void ControllerService::OnDomainUpdate(const DomainUpdateEvent& event) {
    PublicKey pubkey = event.pubkey;
    StabilizePublicKey stabilizing_pubkey;
    auto ret = stabilizing_pubkey.FromBytes(event.stabilizing_pubkey);
    NodeId node_id = mycrypto::Digester::Digest(pubkey).substr(0,
NODE_ID_LENGTH);
    NodeRole node_role = static_cast<NodeRole>(event.domain_role);
    BlockNumber block_num = event.block_num;
    BlockNumber effective_block_num = event.effective_block_num;
    if (!ret.ok()) {
        LOGF_ERROR(
            'Invalid aggr pubkey, node id: {}, pubkey: {}, aggr pubkey:
{}, block_num: {}, '
            'effective_block_num: {}',
            event.domain_id,
            pubkey,
            event.stabilizing_pubkey,
            block_num,
            effective_block_num);
    }

    // only update consensus role
    if (node_role == NodeRole::NODE_ROLE_CONSENSUS) {
        if (event.domain_state == ContractNodeState::NODE_STATE_NORMAL) {
            domain_cache->ChangeValidators(effective_block_num,
                                           node_id,
                                           pubkey,
                                           stabilizing_pubkey);
        }
    }
    //@audit Invalid key added to validator set
}
```

```
    }  
    // ...  
}  
}
```

RECOMMENDATIONS:

Add proper error handling to prevent processing of invalid BLS keys:

```
void ControllerService::OnDomainUpdate(const DomainUpdateEvent& event) {  
    PublicKey pubkey = event.pubkey;  
    StabilizePublicKey stabilizing_pubkey;  
    auto ret = stabilizing_pubkey.FromBytes(event.stabilizing_pubkey);  
    if (!ret.ok()) {  
        LOGF_ERROR(  
            'Invalid aggr pubkey, node id: {}, pubkey: {}, aggr pubkey:  
{}, block_num: {}, '  
            'effective_block_num: {}',  
            event.domain_id,  
            pubkey,  
            event.stabilizing_pubkey,  
            block_num,  
            effective_block_num);  
+         return; // Exit early on invalid BLS key  
    }  
  
    // ...  
}
```


3.60 Low-Configuration Validation Error Handling Inconsistency

SEVERITY:

LOW

STATUS:

Fixed

DESCRIPTION:

The `InitializeImpl()` method in `aldaba-ng/services/controller/controller_service.cpp` uses chained `&&` operations for configuration validation but logs a hardcoded configuration key in error messages. This creates debugging difficulties as the logged key may not match the actual failed configuration item due to short-circuit evaluation.

```
bool ret = conf::GetJsonMember(service_root,
                                kRpcConsensusThreadNumberKey,
                                &rpc_consensus_thread_num,
                                &rpc_consensus_thread_num)
    && conf::GetJsonMember(service_root,
                            kRpcProposeThreadNumberKey,
                            &rpc_propose_thread_num,
                            &rpc_propose_thread_num)
    && conf::GetJsonMember(service_root,
                            kRpcSyncThreadNumberKey,
                            &rpc_sync_thread_num,
                            &rpc_sync_thread_num)
    && conf::GetJsonMember(service_root,
                            kProposeQueueDepth,
                            &propose_queue_depth,
                            &propose_queue_depth)
    && conf::GetJsonMember(service_root,
                            kTxCacheQueueSize,
                            &txcache_queue_size,
                            &txcache_queue_size)
    && conf::GetJsonMember(service_root,
                            kSimulateQueueSize,
                            &simulate_queue_size,
                            &simulate_queue_size);

if (!ret || (rpc_consensus_thread_num < 1 || rpc_propose_thread_num < 1
            || rpc_sync_thread_num < 1)) {
    LOG_KV_ERROR(sLogger,
                 ('Initialize parse failed, key',
                  kRpcSyncThreadNumberKey) // Hardcoded key
```

```
        ('Config', service_conf));  
    return false;  
}
```

Scenarios:

- If `rpc_consensus_thread_num` fails validation, the log shows ' 'rpc_sync_thread_num' '
- If `simulate_queue_size` fails validation, the log still shows ' 'rpc_sync_thread_num' '

RECOMMENDATIONS:

Implement proper error tracking to identify the specific failed configuration item.

3.61 Low-DOG Service Resource Cleanup Failure on Initialization Error

SEVERITY:

LOW

STATUS:

Fixed

PATH:

aldaba-ng/services/dog/dog_service.cpp

DESCRIPTION:

`DOGService::InitializeImpl()` fails to properly clean up allocated resources when initialization fails. The upper-layer `Service::Shutdown()` method only calls `ShutdownImpl()` when `started_` flag is true, but this flag is only set after successful `Startup()`, leaving resources allocated during `InitializeImpl()` uncleaned.

```
// aldaba-ng/services/dog/dog_service.cpp

void Service<ServiceCls, service>::Shutdown() {
    UnRegisterSelf();
    if (started_) {
        ShutdownImpl();
    }
}
```

RECOMMENDATIONS:

Cleanup on initialization failure:

```
template <typename ServiceCls, ServiceEnum service>
int Service<ServiceCls, service>::Initialize() {
    try {
        if (!InitFootstone()) {
            printf('Failed to init footstone for service %s\n',
name_.c_str());
+           ShutdownImpl();
            return 7;
        }

        if (!InitializeImpl()) {
            printf('Failed to initialize service impl.\n');
        }
    }
}
```

```
+         ShutdownImpl();
            return 8;
        }
    } catch (const std::exception& e) {
        printf('std exeception: %s\n', e.what());
+         ShutdownImpl();
            return 255;
        }
        return 0;
    }
```

3.62 Low-DOG Service Division by Zero in Consensus Node List Access

SEVERITY:

LOW

STATUS:

Fixed

PATH:

aldaba-ng/services/dog/dog_service.cpp

DESCRIPTION:

`GetForwardTransactionRank()` and `ComputeForwardTarget()` perform modulo operation on `consensus_node_list_.size()` without checking if the list is empty, leading to division by zero when `consensus_node_list_.size() == 0`.

```
// aldaba-ng/services/dog/dog_service.cpp

bool DOGService::GetForwardTransactionRank(...) {
    uint64_t index = hash_num % consensus_node_list_.size();
    for (size_t i = 0; i < 3; i++) {
        auto target = consensus_node_list_[(index + i) %
consensus_node_list_.size()].node_id_;
    }
}

bool DOGService::ComputeForwardTarget(...) {
    uint64_t index = hash_num % consensus_node_list_.size();
    for (size_t i = 0; i < 3; i++) {
        auto target = consensus_node_list_[(index + i) %
consensus_node_list_.size()].node_id_;
    }
}
```

RECOMMENDATIONS:

Add empty list check before modulo operation.

```
bool DOGService::GetForwardTransactionRank(...) {
+   if (consensus_node_list_.empty()) {
+       return false;
+   }
```

```
uint64_t index = hash_num % consensus_node_list_.size();
for (size_t i = 0; i < 3; i++) {
    auto target = consensus_node_list_[(index + i) %
consensus_node_list_.size()].node_id_;
}
}

bool DOGService::ComputeForwardTarget(...) {
+   if (consensus_node_list_.empty()) {
+       return false;
+   }
    uint64_t index = hash_num % consensus_node_list_.size();
    for (size_t i = 0; i < 3; i++) {
        auto target = consensus_node_list_[(index + i) %
consensus_node_list_.size()].node_id_;
    }
}
```

3.63 Low-Race Condition in BroadcastMessage Leading to Resource Limit Bypass

SEVERITY:

LOW

STATUS:

Fixed

DESCRIPTION:

`BroadcastMessage()` contains a TOCTOU race condition where the node count check and the actual broadcast operation are separated by a time window, allowing concurrent threads to bypass connection limits.

```
bool FcManager::BroadcastMessage(std::shared_ptr<PacketWrapper> packet,
    NodeRole role) {
    std::lock_guard<std::mutex> guard(nodes_mutex_);
    if (node_manager_->GetNodeSize() == 0) {
        LOGF_TRACE('No node with role = {} ', role);
        return false;
    }

    LOGF_TRACE('fc broadcast message[type : {}] to node[node role {}]',
        packet->msg_type_, role);
    node_manager_->BroadcastMessage(packet, role);

    return true;
}

bool FcManager::IsConnectable(NodeId& node_id, bool incoming) {
    // ...

    // non consensus node has connection limit
    if (node_info.node_role_ == NodeRole::NODE_ROLE_NONCONSENSUS
        && GetConnectedNodeSize(NodeRole::NODE_ROLE_NONCONSENSUS) >=
        max_nonconsensus_session_) {
        LOGF_WARN('Non consensus node has exceed the session limit {} ',
            max_nonconsensus_session_);
        return false;
    }

    return true;
}
```

RECOMMENDATIONS:

Use atomic operations or single critical section to eliminate race condition.

3.64 Low-Session Rate Limiting Config Parameter Mapping Error

SEVERITY:

LOW

STATUS:

Fixed

PATH:

aldaba-ng/services/portal/portal_config.cpp

DESCRIPTION:

Incorrect parameter mapping in `SessionManagerConfig::Initialize()` where session and thread PPS rate limiting values are swapped, causing rate limiting configuration to be applied to wrong targets.

```
// aldaba-ng/services/portal/portal_config.cpp
&& GetJsonMember(root,
                  gMaxInboundPpsPerThread,
                  &max_inbound_pps_per_session_,
                  &max_inbound_pps_per_session_)
&& GetJsonMember(root,
                  gMaxInboundPpsPerSession,
                  &max_inbound_pps_per_thread_,
                  &max_inbound_pps_per_thread_)
```

Same issues:

```
    && GetJsonMember(root,
                    gDebugWorkerCount,
                    &subscribe_worker_count_,
                    &subscribe_worker_count_)
    && GetJsonMember(root,
                    gDebugWorkerQueueSize,
                    &debug_worker_queue_size_,
                    &debug_worker_queue_size_)
    && GetJsonMember(root, gEthDefaultWorkerCount,
                    &debug_worker_count_, &debug_worker_count_)
```

RECOMMENDATIONS:

Swap the assignment targets to correct the mapping:

```
- && GetJsonMember(root, gMaxInboundPpsPerThread,  
    &max_inbound_pps_per_session_, &max_inbound_pps_per_session_)  
+ && GetJsonMember(root, gMaxInboundPpsPerThread,  
    &max_inbound_pps_per_thread_, &max_inbound_pps_per_thread_)  
- && GetJsonMember(root, gMaxInboundPpsPerSession,  
    &max_inbound_pps_per_thread_, &max_inbound_pps_per_thread_)  
+ && GetJsonMember(root, gMaxInboundPpsPerSession,  
    &max_inbound_pps_per_session_, &max_inbound_pps_per_session_)  
  
// ...
```

3.65 Low-HealthChecker Indefinite Wait Causes DoS on Unresponsive RPC

SEVERITY:

LOW

STATUS:

Acknowledge

PATH:

aldaba-ng/libraries/thread/countdown.cpp\aldaba-ng/services/portal/health_checker

DESCRIPTION:

Calling `CountDownLatch::Wait()` on a single-threaded I/O context. `CountDownLatch::Wait()` has no timeout. If any txpool RPC callback is never invoked, the latch never reaches zero and the checker thread stalls permanently.

```
// aldaba-ng/libraries/thread/countdown.cpp
void CountDownLatch::Wait() {
    UniqueGuard lock(mutex_);
    while (count_ > 0) {
        condition_.wait(lock);
    }
}
```

```
// aldaba-ng/services/portal/health_checker.cpp
void HealthChecker::PrepareCheckHealth() {
    // ...

    if (checking_) {
        return;
    }
    checking_ = true;

    std::atomic<bool> check_result{true};
    std::atomic<uint32_t> dispatch_tx_size{0};
    std::atomic<uint32_t> dispatch_tx_limit{0};

    std::vector<std::string> endpoints =
txpool_client_>GetServiceEndpoints();
    CountDownLatch cd(static_cast<int>(endpoints.size()));
    // send request to all txpool instance
    for (const auto& ep : endpoints) {
```

```
LOGF_DEBUG('GetHealth begin query txpool instance {}', ep);
// use txpool transaction count to calculate `load`
schema::GetServiceStatusRequest req;
req.request_type = 0; // place_holder
auto call_context = new GetServiceStatusCallRPCContext(&req);
call_context->done_ =
    ALDABA_CAST_TO_RPC_CLOSURE(NewClosure(this,

&HealthChecker::ReceiveTxpoolHealth,

                                call_context,
                                &dispatch_tx_size,
                                &dispatch_tx_limit,
                                &check_result,
                                &cd));

    txpool_client_->GetServiceStatus(ep, call_context);
}
cd.Wait();
dispatch_tx_size_ = dispatch_tx_size.load();
dispatch_tx_limit_ = dispatch_tx_limit.load();
last_check_result_ = check_result.load();
checking_ = false;
}
```

RECOMMENDATIONS:

If wait times out, log and proceed to the next scheduled health check rather than blocking indefinitely.

```
- cd.Wait();
+ if (!cd.WaitFor(std::chrono::milliseconds(FLAGS_rpc_timeout_ms))) {
+     LOG_PTL_WARN('health check timeout');
+ }
```

3.66 Low-ETH Subscribe Logs DoS

SEVERITY:

LOW

STATUS:

Acknowledge

PATH:

aldaba-ng/services/portal/dispatcher.cpp

DESCRIPTION:

The `eth_subscribe` implementation allows clients to create subscriptions with overly broad filters. When filters are empty (`address_bytes.size() == 0` or `topics_bytes.size() == 0`), bloom filter check always returns true, multiple broad subscriptions cause disk I/O and CPU exhaustion on every new block.

```
// aldaba-ng/services/portal/dispatcher.cpp
    Json::Value data(Json::arrayValue);
    schema::Block block{};
    auto ret =
factory_->GetChainProvider()->ReadBlock(block_header.number, &block);
    if (ret != StorageHandleCode::SUCCESS) {
        ServeError(ctx,
                    JSONRPCError::SERVER_ERROR,
                    'QUERY_CLIENT_ERROR: failed to read block body');
        return;
    }
```

RECOMMENDATIONS:

1. Limit subscription count per session
2. Require valid filters (address or topic)
3. Use indexed queries instead of full block reads

ZAN RPC WS Limit?

3.67 Low-Internal RPC Call Missing Timeout Handling

SEVERITY:

LOW

STATUS:

Acknowledge

PATH:

aldaba-ng/services/compute/compute_service.cpp\aldaba-ng/libraries/thread/countdown.cpp

DESCRIPTION:

`ComputeService` uses `CountDownLatch::Wait()` without timeout mechanism when making internal RPC calls to dependent services. worker threads to block indefinitely when dependent services become unresponsive.

```
// aldaba-ng/services/compute/compute_service.cpp
void ComputeService::FetchTransaction(CUExecuteRPCContextRawPtr
    exec_context) {
    CETINA_SCOPE_EXIT_DURATION_NS(TXGROUP_FETCH_DURATION);
    CU_LOGF_DEBUG('begin fetch transactions from txpool');

    exec_context->sync_ = std::make_unique<CountDownLatch>(1);
    auto dispatch_context = new EVMFetchTransactionDispatchRPCContext;
    dispatch_context->done_ = ALDABA_CAST_TO_RPC_CLOSURE(
        NewClosure(this, &ComputeService::FetchTransactionDone,
            exec_context, dispatch_context));
    FetchTransactionCallMapper
    call_mapper(txpool_client_->GetComboChannel(),

    txpool_client_->GetPartitions(),
                                exec_context);
    txpool_client_->FetchTransaction(dispatch_context, &call_mapper);
    exec_context->sync_->Wait();

    CETINA_DURATION_BEGIN(exec_context->end_fetch_timestamp_ns_);
    CU_LOGF_DEBUG('end fetch transactions from txpool');
}
```

```
// aldaba-ng/libraries/thread/countdown.cpp
void CountDownLatch::Wait() {
    UniqueGuard lock(mutex_);
```

```
while (count_ > 0) {  
    condition_.wait(lock);  
}  
}
```

RECOMMENDATIONS:

Implement timeout mechanism for all blocking RPC calls.

```
- exec_context->sync_->Wait();  
+ if (!exec_context->sync_->WaitFor(std::chrono::seconds(30))) {  
}
```

3.68 Low-SSZViews Function Out-of-Bounds Read

SEVERITY:

LOW

STATUS:

Fixed

PATH:

aldaba-ng/libraries/rpc/rpc_helper.h\aldaba-ng/libraries/common/aldaba_buffer.h\

DESCRIPTION:

`SSZViews` performs out-of-bounds memory reads when processing malformed RPC requests with length less than 4 bytes.

```
// aldaba-ng/libraries/rpc/rpc_helper.h
std::pair<ssz::SSZObjectView<T>, ssz::SSZObjectView<schema::RpcMeta>>
inline SSZViews(
    const ABuffer& msg_body) {
    uint32_t meta_len = 0;
    msg_body.GetInt(0, meta_len); // out of bound
    // validation after read
    if (msg_body.Size() <= META_HEAD + meta_len) {
        return {ssz::SSZObjectView<T>(ssz::Buffer()),
                ssz::SSZObjectView<schema::RpcMeta>(ssz::Buffer())};
    }
    // ...
}
```

```
// aldaba-ng/libraries/common/aldaba_buffer.h
template <typename T>
size_t ABuffer::GetInt(size_t offset, T& t) const {
    t = 0;
    for (size_t i = 0; i < sizeof(T); ++i) { // sizeof(uint32_t) = 4
        int shift = 8 * (sizeof(T) - i - 1);
        t = t | (((T)(head_[offset + i])) << shift); // Out-of-bounds
    }
    return offset + sizeof(T);
}
```


RECOMMENDATIONS:

Add buffer size validation before reading the length field.

3.69 Low-ComboChannel::AddServer Null Pointer Dereference

SEVERITY:

LOW

STATUS:

Fixed

PATH:

aldaba-ng/libraries/rpc/combo_channel.cpp

DESCRIPTION:

`ComboChannel::AddServer` directly uses the return value of `io_->OpenChannel()` without null pointer checking. a TODO comment acknowledging this issue but fails to implement error handling.

```
// aldaba-ng/libraries/rpc/combo_channel.cpp
for (size_t i = 0; i < s->server_io_thread_num_; ++i) {
    // TODO open failed?
    arpc::EasyRPCChannel* channel =
        io_->OpenChannel(s->endpoint_, ERPC_CONN_IDLE_TIME, false, 1024);
    arpc::ChannelOption option;
    option.policy = 'fastpath';
    channel->Init(option);
    channel->SetPeerThread(i, s->server_io_thread_num_);
    channel->SetConnIdleTime(60 * 60 * 60 * 1000);
    channel->SetTraceFlag(true);
    // ...
    s->channels_.push_back(RPCChannelWrapper{channel, cidx});
}
```

RECOMMENDATIONS:

Implement null pointer checking and error handling.

3.70 Low-Undefined Behavior in Metadata Processing

SEVERITY:

LOW

STATUS:

Fixed

PATH:

aldaba-ng/components/query/query_component_impl.cpp

DESCRIPTION:

ReadMetaData return value is not checked, undefined behavior when accessing empty strings during metadata processing.

```
// aldaba-ng/components/query/query_component_impl.cpp
std::string value;
uint64_t seq_num = 0;
storage_>ReadMetaData(key, value);
// last block in seq
if (value.back() == gLastBlockInSeqSuffix) {
    value.pop_back();                // pop suffix
    sync_block.is_last_block_in_seq = 1; // set last block
    if (value.back() == gLastBlockInSeqSuffix) { // UB if value is empty
        sync_block.is_epoch_end = 1; // set epoch block
        value.pop_back();
    }
}
```

RECOMMENDATIONS:

```
+ if (storage_>ReadMetaData(key, value) != StorageHandleCode::SUCCESS) {
+     // ...
+ } else if (!value.empty() && value.back() == gLastBlockInSeqSuffix) {
- if (value.back() == gLastBlockInSeqSuffix) {
    value.pop_back();
    sync_block.is_last_block_in_seq = 1;
- if (value.back() == gLastBlockInSeqSuffix) {
+ if (!value.empty() && value.back() == gLastBlockInSeqSuffix) {
    sync_block.is_epoch_end = 1;
    value.pop_back();
}
```

}

3.71 Low-RecvBlockProof Unauthenticated Block Proof Acceptance

SEVERITY:

LOW

STATUS:

Acknowledge

PATH:

aldaba-ng/services/controller/controller_service.cpp

DESCRIPTION:

`RecvBlockProof` TODO comment indicates missing sender and validator matching verification.

```
// aldaba-ng/services/controller/controller_service.cpp
void ControllerService::HandleRPCRecvBlockProof(
    RPCRequest<schema::SendBlockProofRequest> rpc_request,
    RecvBlockProofRPCResponse rpc_response_cb) {
    if (IsLegacyStabilizer()) {
        // ...
        std::unique_ptr<schema::RecvBlockProofResponse> response =
            std::make_unique<schema::RecvBlockProofResponse>();
        response->result = 0;
        rpc_response_cb(std::move(response));
        // TODO check sender and validator_set matching
        std::lock_guard<std::mutex> lock(stabilizer_mutex_);
        auto ret = block_stabilizer_->OnBlockProofMsg(bv_ptr);
    }
}
```

RECOMMENDATIONS:

```
void ControllerService::HandleRPCRecvBlockProof(
    RPCRequest<schema::SendBlockProofRequest> rpc_request,
    RecvBlockProofRPCResponse rpc_response_cb) {
    if (IsLegacyStabilizer()) {
+       // Validate sender and validate before processing
+       // ...
+
        // ...
        std::unique_ptr<schema::RecvBlockProofResponse> response =
            std::make_unique<schema::RecvBlockProofResponse>();
    }
}
```

```
        response->result = 0;
        rpc_response_cb(std::move(response));
        // TODO check sender and validator_set matching

        std::lock_guard<std::mutex> lock(stabilizer_mutex_);
        auto ret = block_stabilizer_->OnBlockProofMsg(bv_ptr);
    }
}
```

3.72 Low-Spec Version Misuse in SSZ Decoding

SEVERITY:

LOW

STATUS:

Acknowledge

PATH:

aldaba-ng/services/controller/controller_service.cpp\aldaba-ng/services/controller

DESCRIPTION:

`GetVersionByBlockHeader` incorrectly returns latest spec version for historical blocks, and consensus components consistently use hardcoded `version=0` for `ConsensusContent/ConsensusTx` SSZ operations.

```
// aldaba-ng/services/controller/controller_service.cpp
bool ControllerService::GetVersionByBlockHeader(const uint64_t&
    block_number,
                                           uint16_t* version,
                                           uint64_t* block_timestamp,
                                           std::string* random_seed)
{
    // TODO: Not support sim transaction on history block, because can't
    // get spec version
    // by history block number
    if (!VersionManager::GetInstance()->GetLatestSpecVersion(version)) {
        LOGF_ERROR('failed to get latest spec version');
        return false;
    }
    return true;
}
```

```
// aldaba-ng/services/controller/controller_service.cpp
void ControllerService::HandleRPCSendTxHashList(
    RPCRequest<schema::SendTxHashListRequest> rpc_request,
    SendTxHashListRPCResponse rpc_response_cb) {
    uint16_t ssz_version = 0;
    uint16_t spec_version{0}; // Initialized to 0
    auto epoch_num = rpc_request.req_view_.EpochNumber();
    // ...
    auto buf = tx_info_view.ConsensusTxView().GetBuffer();
}
```

```
    auto consensus_tx = SSZUnPack<schema::ConsensusTx>(buf,  
    spec_version); // spec_version == 0  
}
```

```
// aldaba-ng/services/controller/helper.hpp  
static Proposal Txs2Proposal(  
    const std::vector<std::shared_ptr<controller::TxContext>>& tx_list) {  
    // ...  
    auto result = SSZPack<schema::ConsensusContent>(consensus_content,  
    0); // Hardcoded 0  
    // ...  
}  
  
static size_t AppendProposal2Txs(uint64_t block_number,  
    Proposal proposal,  
    std::string proposer_id,  
  
    std::vector<std::shared_ptr<controller::TxContext>>& tx_list) {  
    // ...  
    auto consensus_tx = SSZUnPack<schema::ConsensusTx>(buf, 0); //  
    Hardcoded 0  
    // ...  
}
```

RECOMMENDATIONS:

Use proper spec version resolution by block number/epoch instead of hardcoded 0 or latest version for all SSZ operations.

3.73 Low-Incomplete State Synchronization in TxpoolService

SEVERITY:

LOW

STATUS:

Acknowledge

PATH:

aldaba-ng/services/txpool/txpool_service.cpp

DESCRIPTION:

`OnDomainStateChange` receives external domain state updates but fails to properly parse and use the controller status from the received data. TODO now use hardcoded `SERVING` instead of parsing `domain_state`.

```
// aldaba-ng/services/txpool/txpool_service.cpp
void TxpoolService::OnDomainStateChange(const std::string& domain_state) {
    LOG_CALL_INFO(sLogger, (kService, Name()));
    {
        WLockGuard _(rw_lock_);
        domain_status_ = domain_state;
    }

    // TODO(zifu): parse form controller status
    std::string controller_status = 'SERVING'; //TODO hardcoded

    // changes between SYNC_STARTING/SERVING
    if (controller_status == 'SERVING') {
        UpdateStatus(RunningStatus::SERVING);
    } else if (controller_status == 'SYNC_STARTING') {
        UpdateStatus(RunningStatus::SYNC_STARTING);
    } else {
        LOG_KV_WARN(sLogger,
                    ('OnDomainStateChange warn', 'other status')
                    ('DomainState', domain_state));
    }
}
```

RECOMMENDATIONS:

Implement proper parsing of controller status from the `domain_state` parameter.

3.74 Low-Incorrect Metrics Counting in EnqueueProposeTx

SEVERITY:

LOW

STATUS:

Fixed

PATH:

aldaba-ng/services/txpool/rpc_service_impl.cpp

DESCRIPTION:

logs an error when `dedup_manager_ -> InsertTransaction(txs)` returns false, but still executes `CETINA_GAUGE_ADD(PUSH_DEQUE_QPS, txs.size())`.

```
void TxpoolService::EnqueueProposeTx(std::vector<TxContextSharedPtr>&&
txs) {
    if (!dedup_manager_ -> InsertTransaction(txs)) {
        LOG_KV_ERROR(sLogger,
                     ('ProposeTx failed', 'dedup manager reject')
                     ('ServerStatus',
                      static_cast<uint8_t>(GetStatus())));
    }
    CETINA_GAUGE_ADD(PUSH_DEQUE_QPS, txs.size());
}
```

For example, in RPC service implementation, success QPS and failure counters are properly separated.

```
// aldaba-ng/services/txpool/rpc_service_impl.cpp
    if (ret != ErrorCode::SUCCESS) {
        LOG_KV_ERROR(sLogger,
                     (__FUNCTION__, 'fail')
                     ('Detail info', 'PrepareAccounts failed'));

        rpc_gu.UpdateResult(ErrorCode::COMPUTE_STORAGE_NEED_REOPEN,
                           'PrepareAccounts failed');
        CETINA_GAUGE_ADD(RPC_DISPATCH_FAILED_COUNTER, 1);
        return;
    }
    CETINA_DURATION_END(DISPATCH_PREPARE_ACCOUNTS_LATENCY,
latency);
```

RECOMMENDATIONS:

Only increment the success metric when the dedup manager successfully inserts transactions or add a separate failure counter.

```
void TxpoolService::EnqueueProposeTxs(std::vector<TxContextSharedPtr>&&
    txs) {
    if (!dedup_manager_->InsertTransaction(txs)) {
        LOG_KV_ERROR(sLogger,
                     ('ProposeTx failed', 'dedup manager reject')
                     ('ServerStatus',
                      static_cast<uint8_t>(GetStatus())));
+       return;
    }
    CETINA_GAUGE_ADD(PUSH_DEQUE_QPS, txs.size());
}
```

3.75 Low-Inconsistent Boundary Condition in HandleMetricsCollect

SEVERITY:

LOW

STATUS:

Fixed

PATH:

aldaba-ng/services/txpool/txpool_service.cpp

DESCRIPTION:

Inconsistent boundary condition between the comment and implementation. comment states `LE 100` (Less than or Equal to 100) but the code uses strict less than comparison.

```
// aldaba-ng/services/txpool/txpool_service.cpp
// dump remain txs if running tx LE 100
if (FLAG(enable_dump_remain_txs) && running_tx && running_tx < 100) {
    std::vector<std::string> remains;
    for (auto& kv : partition_ctxs_) {
        auto& pctx = kv.second;
        if (pctx) {
            pctx->DumpRemainTxs(remains);
        }
    }
    LOG_KV_ERROR(sLogger,
        ('RemainTxsSize', remains.size())
        ('RunningTxsSize', running_tx)
        ('RemainTxs', remains));
}
```

RECOMMENDATIONS:

```
// dump remain txs if running tx LE 100
- if (FLAG(enable_dump_remain_txs) && running_tx && running_tx < 100) {
+ if (FLAG(enable_dump_remain_txs) && running_tx && running_tx <= 100) {
```

3.76 Low-Incorrect Shutdown Status Semantics in IsShutdown()

SEVERITY:

LOW

STATUS:

Fixed

PATH:

aldaba-ng/services/txpool/common.h

DESCRIPTION:

`IsShutdown()` includes `STOP_FAILED` status as a `shutdown` state should not be considered as `shutdown` since resources may not be properly cleaned up.

```
// aldaba-ng/services/txpool/common.h
bool IsShutdown() {
    auto status = status_.load();
    return status == RunningStatus::STOPPED || status ==
RunningStatus::STOPPING
        || status == RunningStatus::STOP_FAILED;
}

bool Shutdown() {
    if (IsShutdown()) {
        return false;
    }

    status_.store(RunningStatus::STOPPING);
    bool ret = this->ShutdownImpl();
    if (ret) {
        status_.store(RunningStatus::STOPPED);
    } else {
        status_.store(RunningStatus::STOP_FAILED);
    }

    return ret;
}
```

If stop failed, `Shutdown()` will return false always.

RECOMMENDATIONS:

```
// aldaba-ng/services/txpool/common.h
bool IsShutdown() {
    auto status = status_.load();
    return status == RunningStatus::STOPPED || status ==
        RunningStatus::STOPPING
-         || status == RunningStatus::STOP_FAILED;
}
```

3.77 Low-Unimplemented Partition Management Functions

SEVERITY:

LOW

STATUS:

Fixed

PATH:

aldaba-ng/services/txpool/partition_manager.cpp

DESCRIPTION:

`LoadPartition` and `UnloadPartition` return success without performing any actual operations.

```
// aldaba-ng/services/txpool/partition_manager.cpp
ReturnCode PartitionManager::LoadPartition(uint8_t partition) {
    return ReturnCode(); // Returns but does nothing
}

ReturnCode PartitionManager::UnloadPartition(uint8_t partition) {
    return ReturnCode(); // Returns but does nothing
}
```

RECOMMENDATIONS:

Implement the actual functionality or return appropriate error codes.

```
ReturnCode PartitionManager::LoadPartition(uint8_t partition) {
+   // TODO: Implement
+   LOG_KV_WARN(sLogger, ('LoadPartition not implemented', partition));
+   return ReturnCode(ReturnCode::kNotSupported, 'LoadPartition not
    implemented');
-   return ReturnCode();
}
```

3.78 Low-Infinite Loop in Process Termination

SEVERITY:

LOW

STATUS:

Acknowledge

PATH:

aldaba-ng/libraries/app/app.cpp

DESCRIPTION:

while loop waits indefinitely for process termination without timeout protection. If the target process is unresponsive or cannot be killed by SIGTERM, the application will hang forever, causing dos.

```
// aldaba-ng/libraries/app/app.cpp
if (cli_params_.stop_ || cli_params_.restart_) {
    // ...
    if (kill(pid, SIGTERM) == -1) {
        printf('restart or stop %s failed, add error is %s \n',
            name_.c_str(), strerror(errno));
    }
    int count = 0;
    while (!kill(pid, 0)) { // No timeout check
        std::this_thread::sleep_for(std::chrono::milliseconds(1000));
        printf(' %s[%d] still exists...\n', name_.c_str(), pid);
        count++; // Counter exists but unused
    }
    printf(' %s killed.\n', name_.c_str());
}
```

RECOMMENDATIONS:

```
- int count = 0;
- while (!kill(pid, 0)) {
-     std::this_thread::sleep_for(std::chrono::milliseconds(1000));
-     printf(' %s[%d] still exists...\n', name_.c_str(), pid);
-     count++;
- }
+ int count = 0;
+ const int MAX_WAIT_SECONDS = 30;
+
```



```
+ while (!kill(pid, 0) && count < MAX_WAIT_SECONDS) {  
+     std::this_thread::sleep_for(std::chrono::milliseconds(1000));  
+     printf(' %s[%d] still exists... (%d/%d)\n',  
+         name_.c_str(), pid, count + 1, MAX_WAIT_SECONDS);  
+     count++;  
+ }  
+  
+ if (count >= MAX_WAIT_SECONDS) {  
+     printf('Force killing %s[%d] after timeout\n', name_.c_str(), pid);  
+     kill(pid, SIGKILL);  
+ }
```

3.79 Low-Silent Failure When ThreadPool Size is Zero

SEVERITY:

LOW

STATUS:

Fixed

PATH:

aldaba-ng/libraries/thread/thread_pool.cpp

DESCRIPTION:

`ThreadPool` constructor accepts `size=0` without validation, no worker threads are created to call `io_context_.run()`. Tasks are posted to the IO context but never executed, causing `ParallelExecuteTasks` to hang indefinitely and `EnqueueImpl` to silently fail.

```
// aldaba-ng/libraries/thread/thread_pool.cpp
ThreadPool::ThreadPool(const std::string& name, size_t size)
    : name_(name), size_(size), task_count_(0) {
    for (size_t i = 0; i < size; ++i) { // No threads created if size=0
        workers_.create_thread([name, i, this]() { io_context_.run(); });
    }
}

bool ThreadPool::EnqueueImpl(std::function<void()> task) {
    io_context_.post([this, task]() { // Task posted but no threads to
        execute
            task();
            task_count--;
    });
    return true; // Returns success but task never runs
}

void ThreadPool::ParallelExecuteTasks(const
    std::vector<std::function<void()>>& tasks) {
    DynamicCountDownLatch cd(tasks.size());
    for (const auto& task : tasks) {
        io_context_.post([&cd, task]() { // Tasks posted but never
            executed
                task();
                cd.CountDown();
        });
    }
}
```

```
cd.Wait(); // Waits forever since no threads execute tasks
}
```

RECOMMENDATIONS:

```
ThreadPool::ThreadPool(const std::string& name, size_t size)
    : name_(name), size_(size), task_count_(0) {
+   if (size == 0) {
+       LOGF_ERROR('ThreadPool size must be > 0: {}', name);
+       throw std::invalid_argument('ThreadPool size must be > 0');
+   }
    for (size_t i = 0; i < size; ++i) {
        workers_.create_thread([name, i, this]() { io_context_.run(); });
    }
}
```

3.80 Low-Use-After-Free Risk in AddRepeating with Auto Delete Callbacks

SEVERITY:

LOW

STATUS:

Fixed

PATH:

aldaba-ng/libraries/thread/timer_queue.cpp

DESCRIPTION:

`AddRepeating` uses `assert(!callback->IsAutoDelete())` to prevent UAF, but `assert` is removed in Release builds, allowing auto delete callbacks to be registered.

```
// aldaba-ng/libraries/thread/timer_queue.cpp
TimerEvent::Id TimerQueue::AddRepeating(int32_t delay_ms,
                                       int32_t interval_ms,
                                       TimerEvent::Callback* callback) {
    assert(!callback->IsAutoDelete());
    // ...
    event.repetitive_ = true;
    event.callback_ = callback;
    return AddEvent(event) ? event.id_ : kInvalidEventId;
}

void TimerQueue::ManageRoutine() {
    // ...
    if (event.repetitive_) {
        event.expiration_ms_ = current_ms + event.interval_ms_;
        AddEvent(event);
    }
    ExecuteOneEvent(event);
}
```

RECOMMENDATIONS:

Replace `assert` with runtime validation that rejects auto delete callbacks.

```
TimerEvent::Id TimerQueue::AddRepeating(int32_t delay_ms,
                                       int32_t interval_ms,
                                       TimerEvent::Callback* callback) {
```

```
-  assert(!callback->IsAutoDelete());  
+  if (callback == nullptr || callback->IsAutoDelete()) {  
+      LOGF_ERROR('AddRepeating requires non-auto-delete callback');  
+      return kInvalidEventId;  
+  }  
+  // ...  
}
```

3.81 Low-Null Pointer Dereference in ChainUtils::GetBasicGas

SEVERITY:

LOW

STATUS:

Fixed

PATH:

aldaba-ng/libraries/utils/chain_utils.cpp

DESCRIPTION:

`ChainUtils::GetBasicGas` accepts a raw pointer `SSZTransactionRawPtr tx` without null validation and immediately dereferences it.

```
// aldaba-ng/libraries/utils/chain_utils.cpp
uint64_t ChainUtils::GetBasicGas(SSZTransactionRawPtr tx, uint16_t
    spec_version) {
    uint64_t basic_gas = NORMAL_TX_GAS;
    auto tx_viewer = GetSSZTxViewer(*tx); // Null pointer dereference if
    tx is nullptr
    auto access_list = tx_viewer.GetAccessList();
    for (size_t i = 0; i < access_list.Size(); i++) {
        basic_gas += ACCESS_LIST_ADDRESS_GAS;
        basic_gas += ACCESS_LIST_STORAGE_GAS *
        access_list[i].storage_keys.Size();
    }
    return basic_gas;
}
```

RECOMMENDATIONS:

```
uint64_t ChainUtils::GetBasicGas(SSZTransactionRawPtr tx, uint16_t
    spec_version) {
+   if (tx == nullptr) {
+       LOGF_ERROR('GetBasicGas called with null pointer');
+       return 0; // or throw std::invalid_argument('tx cannot be null')
+   }
    uint64_t basic_gas = NORMAL_TX_GAS;
    auto tx_viewer = GetSSZTxViewer(*tx);
    auto access_list = tx_viewer.GetAccessList();
    for (size_t i = 0; i < access_list.Size(); i++) {
```

```
        basic_gas += ACCESS_LIST_ADDRESS_GAS;
        basic_gas += ACCESS_LIST_STORAGE_GAS *
access_list[i].storage_keys.Size();
    }
    return basic_gas;
}
```

3.82 Low-Unchecked Vector Access in EVM State Functions

SEVERITY:

LOW

STATUS:

Fixed

PATH:

aldaba-ng/libraries/utils/evm_state.cpp

DESCRIPTION:

`GetDynamicArrayLength` and `GetString` access vector elements without checking if the vector is empty, `std::out_of_range` exceptions and process termination.

```
// aldaba-ng/libraries/utils/evm_state.cpp
bool GetDynamicArrayLength(const Address& contract_address,
                           const Uint256& base_slot,
                           size_t& size) {
    // ...
    if (read_result == nullptr) {
        return false;
    }

    size_t index = 0; // only one ele
    ReadStorageResult result = read_result->at(index); // Potential
    out_of_range
    // ...
}

std::string GetString(const Address& contract_address, const Uint256&
base_slot) {
    // ...
    if (read_result == nullptr) {
        LOGF_ERROR('ReadStorage failed contract {}, slot {}',
contract_address, ToHex(base_slot));
        return '';
    }
    size_t index = 0; // only one ele
    ReadStorageResult result = read_result->at(index); // Potential
    out_of_range
    // ...
}
```


RECOMMENDATIONS:

Add size validation before accessing vector elements to prevent crashes.

```
bool GetDynamicArrayLength(const Address& contract_address,
                           const Uint256& base_slot,
                           size_t& size) {
    // ...
-   if (read_result == nullptr) {
+   if (read_result == nullptr || read_result->empty()) {
        return false;
    }

    size_t index = 0; // only one ele
    ReadStorageResult result = read_result->at(index);
    // ...
}

std::string GetString(const Address& contract_address, const Uint256&
    base_slot) {
    // ...
-   if (read_result == nullptr) {
+   if (read_result == nullptr || read_result->empty()) {
        LOGF_ERROR('ReadStorage failed contract {}, slot {}',
            contract_address, ToHex(base_slot));
        return '';
    }
    size_t index = 0; // only one ele
    ReadStorageResult result = read_result->at(index);
    // ...
}
```

3.83 Low-Inconsistent Return Values in GenerateFromTemplate Methods

SEVERITY:

LOW

STATUS:

Fixed

PATH:

aldaba-ng/libraries/utils/evm_tx_generator.cpp

DESCRIPTION:

`EVMJsonAPI::GenerateFromTemplate` returns **true** in its else branch while all other `GenerateFromTemplate` methods return **false**.

```
// aldaba-ng/libraries/utils/evm_tx_generator.cpp

// Other methods (EVMTransfer, EVMerc20, EVMerc721, EVMRwa) - return false
bool EVMTransfer::GenerateFromTemplate(uint32_t conn_id,
                                       uint32_t rpc_id,
                                       EVMTransactionInfo& template_tx,
                                       EVMTransactionInfo& output) {

    // ...
} else {
    LOGF_WARN('hot support mode {}', output.mode);
    return false;
}

// EVMJsonAPI method - returns true
bool EVMJsonAPI::GenerateFromTemplate(uint32_t conn_id,
                                       uint32_t rpc_id,
                                       EVMTransactionInfo& template_tx,
                                       EVMTransactionInfo& output) {

    // ...
} else {
    LOGF_WARN('Method: {} rpc_params {}',
              output.rpc_method,
              Util::JsonToString(output.rpc_params));
    return true;
}
```

RECOMMENDATIONS:

```
bool EVMJsonAPI::GenerateFromTemplate(uint32_t conn_id,
                                       uint32_t rpc_id,
                                       EVMTransactionInfo& template_tx,
                                       EVMTransactionInfo& output) {
    // ...
} else {
    LOGF_WARN(' Method: {} rpc_params {}',
              output.rpc_method,
              Util::JsonToString(output.rpc_params));
-   return true;
+   return false;
}
```

3.84 Low-Inadequate Error Handling in Rdtsc Function

SEVERITY:

LOW

STATUS:

Fixed

PATH:

aldaba-ng/libraries/utils/time.cpp

DESCRIPTION:

execution and returns a potentially invalid timestamp when `clock_gettime` fails, instead of properly handling the error condition.

```
// aldaba-ng/libraries/utils/time.cpp

static uint64_t Rdtsc(void) {
    struct timespec ts = {0, 0};
    if (::clock_gettime(CLOCK_MONOTONIC, &ts) != 0) {
        std::cout << 'failed to get high resolution timestamp.' <<
        std::endl;
    }
    return uint64_t(ts.tv_sec) * NS_PER_SEC + ts.tv_nsec;
}
```

IMPACT:

Callers cannot distinguish between a valid timestamp of 0 and an error condition.

RECOMMENDATIONS:

```
static uint64_t Rdtsc(void) {
    struct timespec ts = {0, 0};
    if (::clock_gettime(CLOCK_MONOTONIC, &ts) != 0) {
        std::cout << 'failed to get high resolution timestamp.' <<
        std::endl;
+         throw std::runtime_error('Failed to get high resolution
timestamp');
    }
    return uint64_t(ts.tv_sec) * NS_PER_SEC + ts.tv_nsec;
}
```

}

3.85 Low-Missing Exception Handling in JsonToFormatString

SEVERITY:

LOW

STATUS:

Fixed

PATH:

aldaba-ng/libraries/utils/utils.cpp

DESCRIPTION:

`JsonToFormatString` catches all exceptions but fails to provide proper error handling or logging.

```
// aldaba-ng/libraries/utils/utils.cpp
std::string Util::JsonToFormatString(const Json::Value& js) {
    std::string ret;
    try {
        Json::StreamWriterBuilder builder;
        std::unique_ptr<Json::StreamWriter>
            writer(builder.newStreamWriter());
        std::ostringstream os;
        writer->write(js, &os);
        ret = os.str();
    } catch (...) {
        // LOGF_INFO('json writ failed');
    }
    return ret;
}
```

RECOMMENDATIONS:

```
std::string Util::JsonToFormatString(const Json::Value& js) {
    std::string ret;
    try {
        Json::StreamWriterBuilder builder;
        std::unique_ptr<Json::StreamWriter>
            writer(builder.newStreamWriter());
        std::ostringstream os;
        writer->write(js, &os);
        ret = os.str();
    }
```

```
+     } catch (const std::exception& e) {  
+         LOGF_ERROR('JSON serialization failed: {}', e.what());  
+         // Return a default JSON object or handle the error appropriately  
+         ret = '{}';  
+     } catch (...) {  
+         LOGF_ERROR('Unknown error during JSON serialization');  
+         ret = '{}';  
+     }  
+     return ret;  
+ }
```

3.86 Low-Buffer Overflow in Readu_be

SEVERITY:

LOW

STATUS:

Fixed

PATH:

aldaba-ng/libraries/utils/utils.cpp

DESCRIPTION:

[Readu64_be](#), [Readu32_be](#), [Readu16_be](#), and [Readu128_be](#) lack bounds checking and directly access array elements without validating input data length.

```
// aldaba-ng/libraries/utils/utils.cpp
uint64_t Util::Readu64_be(const std::string_view& data) {
    uint64_t value = 0;
    for (auto i = 0; i < 8; ++i) {
        value = value | (((uint64_t)((byte)data[i])) << (56 - (8 * i)));
    }
    return value;
}

uint32_t Util::Readu32_be(const std::string_view& data) {
    uint32_t value = 0;
    for (auto i = 0; i < 4; ++i) {
        value = value | (static_cast<uint32_t>((byte)data[i]) << (24 - (8
* i)));
    }
    return value;
}

uint16_t Util::Readu16_be(const std::string_view& data) {
    return static_cast<uint16_t>(((byte)data[0] << 8) | ((byte)data[1]));
}

__uint128_t Util::Readu128_be(const std::string_view data) {
    __uint128_t value = 0;
    for (auto i = 0; i < 16; ++i) {
        value = value | (((__uint128_t)((byte)data[i])) << (120 - (8 *
i)));
    }
}
```



```
    return value;
}
```

RECOMMENDATIONS:

```
uint64_t Util::Readu64_be(const std::string_view& data) {
+   if (data.size() < 8) {
+       throw std::invalid_argument('Insufficient data for Readu64_be:
+       expected 8 bytes, got ' +
+                               std::to_string(data.size()));
+   }
    uint64_t value = 0;
    for (auto i = 0; i < 8; ++i) {
        value = value | (((uint64_t)((byte)data[i])) << (56 - (8 * i)));
    }
    return value;
}

uint32_t Util::Readu32_be(const std::string_view& data) {
+   if (data.size() < 4) {
+       throw std::invalid_argument('Insufficient data for Readu32_be:
+       expected 4 bytes, got ' +
+                               std::to_string(data.size()));
+   }
    uint32_t value = 0;
    for (auto i = 0; i < 4; ++i) {
        value = value | (static_cast<uint32_t>((byte)data[i]) << (24 - (8
* i)));
    }
    return value;
}

uint16_t Util::Readu16_be(const std::string_view& data) {
+   if (data.size() < 2) {
+       throw std::invalid_argument('Insufficient data for Readu16_be:
+       expected 2 bytes, got ' +
+                               std::to_string(data.size()));
+   }
    return static_cast<uint16_t>(((byte)data[0] << 8) | ((byte)data[1]));
}
```

```
__uint128_t Util::Readu128_be(const std::string_view data) {  
+   if (data.size() < 16) {  
+       throw std::invalid_argument('Insufficient data for Readu128_be:  
+           expected 16 bytes, got '' +  
+               std::to_string(data.size()));  
+   }  
    __uint128_t value = 0;  
    for (auto i = 0; i < 16; ++i) {  
        value = value | (((__uint128_t)((byte)data[i])) << (120 - (8 *  
i)));  
    }  
    return value;  
}
```

3.87 Low-Incorrect Null Pointer Check in SM2 SetPubkey

SEVERITY:

LOW

STATUS:

Fixed

PATH:

aldaba-ng/libraries/crypto/impl/local/openssl_sm2_signer_v1.cpp

DESCRIPTION:

The code creates an `EVP_PKEY_Ptr` but then checks `ec_key_ptr` instead of the newly created `evp_pkey_ptr`.

```
// aldaba-ng/libraries/crypto/impl/local/openssl_sm2_signer_v1.cpp
EC_KEY_Ptr ec_key_ptr(EC_KEY_new());
if (!ec_key_ptr) {
    return STATUS(ErrorCode::CRYPTO_PROVIDER_ERROR, 'open ssl error');
}
ret = EC_KEY_set_group(ec_key_ptr.get(), ec_group_ptr.get());
if (ret != 1) {
    return STATUS(ErrorCode::CRYPTO_PROVIDER_ERROR, 'open ssl error');
}
ret = EC_KEY_oct2key(ec_key_ptr.get(), (const unsigned
char*)key.data(), key.size(), nullptr);
if (ret != 1) {
    return STATUS(ErrorCode::CRYPTO_PROVIDER_ERROR, 'open ssl error');
}

EVP_PKEY_Ptr evp_pkey_ptr(EVP_PKEY_new());
if (!evp_pkey_ptr) {    // should check evp_pkey_ptr
    return STATUS(ErrorCode::CRYPTO_PROVIDER_ERROR, 'open ssl error');
}

ret = EVP_PKEY_set1_EC_KEY(evp_pkey_ptr.get(), ec_key_ptr.get());
if (ret != 1) {
    return STATUS(ErrorCode::CRYPTO_PROVIDER_ERROR, 'open ssl error');
}
```

RECOMMENDATIONS:

```
EVP_PKEY_Ptr evp_pkey_ptr(EVP_PKEY_new());  
- if (!ec_key_ptr) {  
+ if (!evp_pkey_ptr) {  
    return STATUS(ErrorCode::CRYPTO_PROVIDER_ERROR, 'open ssl error');  
}
```

3.88 Info-Gas-Validator Struct Fields Are Not Packed

SEVERITY:

INFO

STATUS:

Fixed

PATH:

contracts/sys_staking.sol:

DESCRIPTION:

The EVM reads from and writes to storage in 32-byte (256-bit) slots. By grouping multiple variables that are smaller than 32 bytes together, they can be “packed” into a single slot, which can lead to significant gas savings.

```
// contracts/sys_staking.sol:
struct Validator {
    string description;           // 1 slot
    string publicKey;            // 1 slot
    string publicKeyPop;         // 1 slot
    string blsPublicKey;         // 1 slot
    string blsPublicKeyPop;      // 1 slot
    string endpoint;             // 1 slot
    uint8 status;                // Occupies a full slot if not packed
    bytes32 poolId;              // 1 slot
    uint256 totalStake;          // 1 slot
    address owner;               // Occupies a full slot if not packed
    uint256 stakeSnapshot;       // 1 slot
    uint256 pendingWithdrawStake; // 1 slot
    uint8 pendingWithdrawWindow; // Occupies a full slot if not packed
}
```

IMPACT:

Every time a `Validator` struct is created or its fields are updated, the contract consumes more gas than theoretically necessary because it uses more `SSTORE` opcodes. The `SSTORE` opcode is one of the most expensive operations in the EVM.

RECOMMENDATIONS:

```
// Recommend re-ordering for packing
struct Validator {
    string description;
    string publicKey;
    string publicKeyPop;
    string blsPublicKey;
    string blsPublicKeyPop;
    string endpoint;
    bytes32 poolId;
    uint256 totalStake;
    uint256 stakeSnapshot;
    uint256 pendingWithdrawStake;
    // The following fields can be packed into a single 32-byte slot
    address owner;           // 20 bytes
    uint8 status;            // 1 byte
    uint8 pendingWithdrawWindow; // 1 byte
}
```

3.89 Info-Gas-Optimizing nested loop code

SEVERITY:

INFO

STATUS:

Fixed

DESCRIPTION:

In `aldaba-ng/sys_contract/artifact_src/solidity/sys_chaincfg.sol`, the nested loop of the `set_config` function puts `keccak256(abi.encodePacked(conf.key))` in the inner loop, which causes it to be repeatedly calculated and consumes unnecessary gas.

```
function set_config(string[] memory keys, string[] memory values)
    external onlyOwner {
        .....
        if (old_config_cps_number != 0) {
            require(configCps[configCps.length - 2].effectiveBlockNum <=
            block.number, 'INVALID_STATE');
            for (uint256 i = 0; i < configCps[configCps.length -
            2].configs.length; i++) {
                Config storage conf = configCps[configCps.length -
            2].configs[i];
                bool found = false;
                for (uint256 j = 0; j < keys.length; j++) {
                    if (keccak256(abi.encodePacked(conf.key)) ==
            keccak256(abi.encodePacked(keys[j]))) {
                        found = true;
                        break;
                    }
                }
                .....
            }
        }
        .....
    }
}
```

IMPACT:

Repeated calculation of `keccak256(abi.encodePacked(conf.key))` will cause unnecessary gas waste

RECOMMENDATIONS:

Put `keccak256(abi.encodePacked(conf.key))` in the outer loop

```
function set_config(string[] memory keys, string[] memory values)
    external onlyOwner {
        .....
        if (old_config_cps_number != 0) {
            require(configCps[configCps.length - 2].effectiveBlockNum <=
            block.number, 'INVALID_STATE');
            for (uint256 i = 0; i < configCps[configCps.length -
            2].configs.length; i++) {
                Config storage conf = configCps[configCps.length -
            2].configs[i];
+               bytes32 confHash = keccak256(abi.encodePacked(conf.key));
                bool found = false;
                for (uint256 j = 0; j < keys.length; j++) {
-                   if (keccak256(abi.encodePacked(conf.key)) ==
+                   if (confHash == keccak256(abi.encodePacked(keys[j]))) {
+                       if (confHash == keccak256(abi.encodePacked(keys[j]))) {
                            found = true;
                            break;
                        }
                    }
                }
                .....
            }
        }
        .....
    }
}
```


3.90 Info-Gas Opt-Inefficient Array Traversal in advanceEpoch Function

SEVERITY:

INFO

STATUS:

Fixed

DESCRIPTION:

Redundant $O(n^2)$ array search operations in validator status processing causing excessive gas consumption during epoch advancement. When validator status is already set to 0 and `pendingWithdrawWindow != 0`, the next epoch's `advanceEpoch` still traverses `activePoolIds` to find deactivated validators.

```
// aldaba-ng/sys_contract/artifact_src/solidity/sys_staking.sol

// Process pending exits - searches activePoolIds even for deactivated
// validators
for (uint i = 0; i < pendingExitPoolIds.length; i++) {
    bytes32 poolId = pendingExitPoolIds[i];
    for (uint j = 0; j < activePoolIds.length; j++) {
        if (activePoolIds[j] == poolId) {
            if (validators[poolId].totalStake < MIN_POOL_STAKE) {
                validators[poolId].status = 0; // Deactivated
                activePoolIds.pop();
            }
            break;
        }
    }
}
```

RECOMMENDATIONS:

Add status check before array search to avoid redundant traversals.

3.91 Info-The Function MatchTopic argument topic is useless in this function. It will always return true

SEVERITY:

INFO

STATUS:

Fixed

DESCRIPTION:

The Function `MatchTopic` argument *topic* is useless in this function. It will always return true

```
bool Log::MatchTopic(const Topic&topic)const {  
    return true;  
}
```

3.92 Info-Useless function

SEVERITY:

INFO

STATUS:

Fixed

DESCRIPTION:

Description

The Mark() function called in `Insert` but the mark() function no logic, so in here we should delete the mark()

```
MemMPTTrieNode* MemMPTTrieLeafNode::Insert(bytesConstRef key, const
std::string& value) {
    Mark();
    if (Contains(key)) {
        value_ = value;
        return this;
    } else {
        MemMPTTrieNode* n = MemMPTTrieNode::NewBranch(key, value,
bytesConstRef(&ext_), value_);
        delete this;
        return n;
    }
}
```

```
void MemMPTTrieNode::Mark() {
    // aone: 2024122300106604540
    // disable Mark() on every insert step, only Digest() once when
    getting Root at last

    // digest_ = Digest();
}
```

3.93 Info–Unused Parameter in function MerkleTree::Root()

SEVERITY: INFO**STATUS:** Acknowledge

DESCRIPTION:

The Parameter spec_version is not used

```
std::string MerkleTree::Root(std::vector<std::string>& hashes, uint16_t
spec_version) {
    if (hashes.empty()) {
        return std::string(32, '\0');
    }
    const size_t leaf_count = hashes.size();
    const size_t adjust_leaf_count = pow(2, ceil(log2(leaf_count)));
    hashes.resize(adjust_leaf_count);

    while (hashes.size() > 1) {
        if (hashes.size() & 1) {
            hashes.push_back(hashes.back());
        }

        for (size_t i = 0; i < hashes.size(); i += 2) {
            hashes[i] += hashes[i + 1];
            if (i) {
                mycrypto::Digester::Digest(hashes[i], hashes[i / 2],
mycrypto::DigestType::SHA256);
            } else {
                std::string out;
                mycrypto::Digester::Digest(hashes[i], out,
mycrypto::DigestType::SHA256);
                hashes[0] = std::move(out);
            }
        }
        hashes.resize(hashes.size() / 2);
    }

    return hashes[0];
}
```

3.94 Info-Replace hard-coded value

SEVERITY:

INFO

STATUS:

Fixed

DESCRIPTION:

In `sys_chaincfg.sol`, `set_config` will check whether `configCps.length` is equal to 3. It is recommended to use constants instead of hard coding.

```
// sys_chaincfg.sol
function set_config(string[] memory keys, string[] memory values)
    external onlyOwner {
        require(keys.length == values.length, '%Vs are not match');
        if (configCps.length == 3) {
            .....
        }
    }
}
```

IMPACT:

Using constants instead of hard-coded values can improve code readability and maintainability.

RECOMMENDATIONS:

```
+   uint256 public constant MAX_CONFIGCPS_LENGTH = 3;

    function set_config(string[] memory keys, string[] memory values)
    external onlyOwner {
        require(keys.length == values.length, '%Vs are not match');
-       if (configCps.length == 3) {
+       if (configCps.length == MAX_CONFIGCPS_LENGTH) {
            delete configCps[0];
            for (uint256 i = 1; i < configCps.length; i++) {
                configCps[i - 1] = configCps[i];
            }
            configCps.pop();
        }
        .....
    }
```

3.95 Info-Incomplete Log Information in StartupImpl

SEVERITY: INFO**STATUS:** Fixed**DESCRIPTION:**

The `StartupImpl()` method in `aldaba-ng/services/controller/controller_service.cpp` contains incomplete and incorrect log information that may mislead operators and make debugging difficult.

```
bool ControllerService::StartupImpl() {  
  
    LOGF_INFO('ControllerService::StartupImp' );  
  
    // Custom SingletonComponents startup order  
  
    #define STARTUP_HANDLER(do_something, failed_do) \  
  
    if (!do_something) {                                \  
  
        failed_do;                                       \  
  
        return false;                                   \  
  
    }  
}
```

`InitializeImpl()` log info message right.

```
bool ControllerService::InitializeImpl() {  
  
    LOGF_INFO('ControllerService::InitializeImpl' );  
  
    Json::Value service_root;  
  
    std::string service_conf;  
  
    if (!config->GetValue(kServiceSectKey, &service_conf)  
  
    || !Util::StringToJson(service_conf, service_root) ||  
        !service_root.isObject()) {  
  
    }  
}
```

```
LOG_KV_ERROR(sLogger,  
  
('Initialize parse failed, key', kServiceSectKey)  
  
('Config', service_conf));  
  
return false;  
  
}
```

RECOMMENDATIONS:

Fix the typo in the log message to match function name.

```
bool ControllerService::StartupImpl() {  
  
-  LOGF_INFO('ControllerService::StartupImp');  
+  LOGF_INFO('ControllerService::StartupImpl');  
  
// Custom SingletonComponents startup order  
  
#define STARTUP_HANDLER(do_somthing, failed_do) \  
  
if (!do_somthing) {                                \  
  
failed_do;                                          \  
  
return false;                                     \  
  
}
```

3.96 Info-NeedBypassFork Unnecessary Call in Production Environment

SEVERITY:

INFO

STATUS:

Fixed

PATH:

aldaba-ng/services/compute/compute_service.cpp

DESCRIPTION:

`NeedBypassFork` is called on every account load operation in production environments, unnecessary performance overhead since it never executes in production but adds computational cost to each call.

```
// aldaba-ng/services/compute/compute_service.cpp
static bool NeedBypassFork(const CUExecuteRPCContextRawPtr exec_context,
                           const evmc::address& account_addr,
                           const uint64_t& chain_id) {
    constexpr uint64_t kForkChainId = 688688; // h_testnet only
    constexpr uint64_t kForkBlockNumber = 14831638;
    constexpr std::string_view kForkAddr = '2eb02c8b9733b240c6fa73ddf1b25f373199c56c';
    return chain_id == kForkChainId && exec_context->BlockNumber() ==
           kForkBlockNumber
           && ToHex(account_addr) == kForkAddr;
}
```

```
void ComputeService::LoadAccounts(...) const {
    const auto& account_addrs = accounts.first;
    const auto& account_results = accounts.second;
    for (size_t i = 0; i != account_results->size(); ++i) {
        const auto& result = account_results->at(i);
        const auto& addr = account_addrs->at(i);
        auto account_addr = EvmConverter::ToEvmcAddress(addr);
        auto spec_version =
            exec_context->tx_group_context->GetSpecVersion();
        if (spec_version >= SPEC_VERSION::SPEC_VERSION_V4) {
            if (result.first == StorageHandleCode::NOT_FOUND
                || NeedBypassFork(exec_context, account_addr, chain_id_))
            {

```



```
        // NOTE: This aligns with
        `ExecutorTxGroupContext::ReadAccountCallback` to set
        // `acc.exist_` to `false`.
        AccountObject acc;
        acc.exist_ = false;
        evmc_accounts[account_addr] = acc;

        CU_LOGF_INFO('read account: {} successfully, got an empty
account',
                    ToHex(account_addr));

        continue;
    }
}
```

RECOMMENDATIONS:

Remove or optimize the function for production environments using conditional compilation.

3.97 Info-std::cout Error Output Stream Inconsistency

SEVERITY:

INFO

STATUS:

Fixed

PATH:

aldaba-ng/services/binary/main.cpp

DESCRIPTION:**std::cout Error Output Stream Inconsistency**

Error messages in the main entry point are output to `stdout` instead of `stderr`, which violates standard Unix conventions and may interfere with output stream processing in production environments.

```
// aldaba-ng/services/binary/main.cpp
int main(int argc, char** argv) {
    try {
        App app;
        if (!app.InitEnv(argc, argv)) {
            std::cout << 'Failed to init env' << std::endl;
            exit(-1);
        }
        std::cout << 'App Initialize succeed' << std::endl;
        // ...
    } catch (std::exception& e) {
        ret = 255;
        printf('got exeception: %s\n', e.what());
    }
}
```

RECOMMENDATIONS:

For code quality and Unix convention compliance.

```
- std::cout << 'Failed to init env' << std::endl;
+ std::cerr << 'Failed to init env' << std::endl;
```

3.98 Info-Inconsistent Module Name in Error Log

SEVERITY:

INFO

STATUS:

Fixed

PATH:

aldaba-ng/services/txpool/txpool_service.cpp

DESCRIPTION:

Error log `ShutdownERSManager` uses function name instead of module name, inconsistent with other shutdown methods.

```
// aldaba-ng/services/txpool/txpool_service.cpp
bool TxpoolService::ShutdownTxPersistManager() {
    LOG_CALL_INFO(sLogger, (kService, Name()));
    if (!tx_persist_mgr_ || !tx_persist_mgr_->Shutdown()) {
        LOG_KV_ERROR(sLogger,
                     ('Shutdown failed, module', 'TxPersistManager'));
        return false;
    }
    tx_persist_mgr_.reset();

    return true;
}

bool TxpoolService::ShutdownERSManager() {
    LOG_CALL_INFO(sLogger, (kService, Name()));
    if (!ers_mgr_ || !ers_mgr_->Shutdown()) {
        LOG_KV_ERROR(sLogger,
                     ('Shutdown failed, module', 'ShutdownERSManager'));
        return false;
    }
    ers_mgr_.reset();

    return true;
}

bool TxpoolService::ShutdownTxDeduplication() {
    LOG_CALL_INFO(sLogger, (kService, Name()));
    if (!dedup_manager_ || !dedup_manager_->Shutdown()) {
```

```
        LOG_KV_ERROR(sLogger,
                      ('Shutdown failed, module', 'TxDeduplication'));
        return false;
    }
    dedup_manager_.reset();

    return true;
}

bool TxpoolService::ShutdownEventObserver() {
    LOG_CALL_INFO(sLogger, (kService, Name()));
    if (!event_observer_ || !event_observer_->Shutdown()) {
        LOG_KV_ERROR(sLogger,
                      ('Shutdown failed, module', 'EventObserver'));
        return false;
    }
    event_observer_.reset();

    for (auto& evt : event_observers_) {
        evt->Unlisten();
    }
    event_observers_.clear();

    for (auto sub_id : sub_ids_) {
        query_component_->UnsubscribeStableBlock(sub_id);
    }

    return true;
}

bool TxpoolService::ShutdownPartionManager() {
    LOG_CALL_INFO(sLogger, (kService, Name()));
    if (!partition_manager_ || !partition_manager_->Shutdown()) {
        LOG_KV_ERROR(sLogger,
                      ('Shutdown failed, module', 'PartitionContext'));
        return false;
    }
    partition_manager_.reset();

    return true;
}
```

RECOMMENDATIONS:

```
bool TxpoolService::ShutdownERSManager() {
    LOG_CALL_INFO(sLogger, (kService, Name()));
    if (!ers_mgr_ || !ers_mgr_>Shutdown()) {
        LOG_KV_ERROR(sLogger,
-           ('Shutdown failed, module', 'ShutdownERSManager'));
+           ('Shutdown failed, module', 'ERSManager'));
        return false;
    }
    ers_mgr_.reset();

    return true;
}
```

3.99 Info-Incorrect Service Name in Process Lock Error Message

SEVERITY:

INFO

STATUS:

Fixed

PATH:

aldaba-ng/libraries/app/app.cpp

DESCRIPTION:

In full mode, `name_` contains the service name (“txpool”, “portal”), but the error message displays `argv[0]` (program name like “./aldaba”). This makes it unclear which service is already running.

```
// aldaba-ng/libraries/app/app.cpp
bool App::InitEnv(int argc, char** argv) {
    // ...
#ifdef LIGHT
    cli11_.add_option('-s,--service', name_, 'service name');
#endif
    // ...
    auto success = Util::LockProcess('' + name_ + '_lock_file');
    if (!success) {
        printf('%s already running?\n', argv[0]); // Uses program name
        return false;
    }
    // ...
}
```

RECOMMENDATIONS:

```
auto success = Util::LockProcess('' + name_ + '_lock_file');
if (!success) {
-     printf('%s already running?\n', argv[0]);
+     printf('%s already running?\n', name_.c_str());
    return false;
}
```

3.100 Info-Invalid Conditional Statement in EVMTxGenerator::GenerateTransaction

SEVERITY:

INFO

STATUS:

Fixed

PATH:

aldaba-ng/libraries/utils/evm_tx_generator.cpp

DESCRIPTION:

`EVMTxGenerator::GenerateTransaction` is invalid as both **if** and **else** branches execute identical operations. make sure the condition is complete or not.

```
// aldaba-ng/libraries/utils/evm_tx_generator.cpp
if (rpc_method == METHOD_SEND_RAW_TRANSACTION || rpc_method ==
    METHOD_ETH_CALL) {
    evm_tx_infos.rpc_method = rpc_method;
} else {
    evm_tx_infos.rpc_method = rpc_method;
}
```

3.101 Info-Incorrect Error Codes in Ciphertext Parsing Functions

SEVERITY:

INFO

STATUS:

Fixed

PATH:

aldaba-ng/libraries/crypto/impl/crypto_data.cpp

DESCRIPTION:

`PkeyCiphertext::FromBytes` and `SymCiphertext::FromBytes` use incorrect error codes that reference signatures instead of ciphertexts.

```
// aldaba-ng/libraries/crypto/impl/crypto_data.cpp
// PkeyCiphertext::FromBytes
if (in.size() <= MYCRYPTO_ID_SIZE + kPubkeyFingerprintSize) {
    return STATUS(ErrorCode::CRYPTO_INVALID_SIGNATURE, 'length of
signature is invalid');
}

if (GET_ALGO_TYPE_ID(id_) != algo_type_id::ASYM || GET_TYPE_ID(id_) !=
type_id::ASYM_CIPHER) {
    return STATUS(ErrorCode::CRYPTO_INVALID_SIGNATURE, 'Invalid Pkey
cipher type');
}

// SymCiphertext::FromBytes
if (in.size() < MYCRYPTO_ID_SIZE) {
    return STATUS(ErrorCode::CRYPTO_INVALID_SIGNATURE,
        'length of ciphertext is invalid, lack of mycrypto id');
}

if (in.size() < (size_t)MYCRYPTO_ID_SIZE + kDefaultTagLen +
kDefaultIVLen) {
    return STATUS(ErrorCode::CRYPTO_INVALID_SIGNATURE,
        'length of ciphertext is invalid, for aes-gcm should be
        '
        'at least Id + IV + Tag');
}
```


RECOMMENDATIONS:

Replace the incorrect error codes with the correct ones and update error messages to match.

```
// aldaba-ng/libraries/crypto/impl/crypto_data.cpp
// PkeyCiphertext::FromBytes
if (in.size() <= MYCRYPTO_ID_SIZE + kPubkeyFingerprintSize) {
-   return STATUS(ErrorCode::CRYPTO_INVALID_SIGNATURE, 'length of
signature is invalid');
+   return STATUS(ErrorCode::CRYPTO_INVALID_CIPHertext, 'length of
ciphertext is invalid');
}

if (GET_ALGO_TYPE_ID(id_) != algo_type_id::ASYM || GET_TYPE_ID(id_) !=
type_id::ASYM_CIPHER) {
-   return STATUS(ErrorCode::CRYPTO_INVALID_SIGNATURE, 'Invalid Pkey
cipher type');
+   return STATUS(ErrorCode::CRYPTO_INVALID_CIPHertext, 'Invalid Pkey
cipher type');
}

// SymCiphertext::FromBytes
if (in.size() < MYCRYPTO_ID_SIZE) {
-   return STATUS(ErrorCode::CRYPTO_INVALID_SIGNATURE,
-               'length of ciphertext is invalid, lack of mycrypto id');
+   return STATUS(ErrorCode::CRYPTO_INVALID_CIPHertext,
+               'length of ciphertext is invalid, lack of mycrypto id');
}

if (in.size() < (size_t)MYCRYPTO_ID_SIZE + kDefaultTagLen +
kDefaultIVLen) {
-   return STATUS(ErrorCode::CRYPTO_INVALID_SIGNATURE,
-               'length of ciphertext is invalid, for aes-gcm should be
''
-               'at least Id + IV + Tag');
+   return STATUS(ErrorCode::CRYPTO_INVALID_CIPHertext,
+               'length of ciphertext is invalid, for aes-gcm should be
''
+               'at least Id + IV + Tag');
}
```

4. CONCLUSION

In this audit, we thoroughly analyzed **Pharos** blockchain implementation. The problems found are described and explained in detail in Section 3. The problems found in the audit have been communicated to the project leader. We therefore consider the audit result to be **PASSED**.

To improve this report, we greatly appreciate any constructive feedbacks or suggestions, on our methodology, audit findings, or potential gaps in scope/coverage.

5. APPENDIX

5.1 Basic Coding Assessment

5.1.1 Apply Verification Control

Description	The security of apply verification
Result	Not found
Severity	CRITICAL

5.1.2 Authorization Access Control

Description	Permission checks for external integral functions
Result	Not found
Severity	CRITICAL

5.1.3 Forged Transfer Vulnerability

Description	Assess whether there is a forged transfer notification vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.4 Transaction Rollback Attack

Description	Assess whether there is transaction rollback attack vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.5 Transaction Block Stuffing Attack

Description	Assess whether there is transaction blocking attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.6 Soft Fail Attack Assessment

Description	Assess whether there is soft fail attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.7 Hard Fail Attack Assessment

Description	Examine for hard fail attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.8 Abnormal Memo Assessment

Description	Assess whether there is abnormal memo vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.9 Abnormal Resource Consumption

Description	Examine whether abnormal resource consumption in contract processing
Result	Not found
Severity	CRITICAL

5.1.10 Random Number Security

Description	Examine whether the code uses insecure random number
Result	Not found
Severity	CRITICAL

5.2 Advanced Code Scrutiny

5.2.1 Cryptography Security

Description	Examine for weakness in cryptograph implementation
Result	Not found
Severity	HIGH

5.2.2 Account Permission Control

Description	Examine permission control issue in the contract
Result	Not found
Severity	MEDIUM

5.2.3 Malicious Code Behavior

Description	Examine whether sensitive behavior present in the code
Result	Not found
Severity	MEDIUM

5.2.4 Sensitive Information Disclosure

Description	Examine whether sensitive information disclosure issue present in the code
Result	Not found
Severity	MEDIUM

5.2.5 System API

Description	Examine whether system API application issue present in the code
Result	Not found
Severity	LOW

6. DISCLAIMER

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to the Company in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes without ExVul's prior written consent.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts ExVul to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. ExVul's position is that each company and individual are responsible for their own due diligence and continuous security. ExVul's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

7. REFERENCES

- [1] MITRE. CWE-191: Integer Underflow (Wrap or Wraparound). <https://cwe.mitre.org/data/definitions/191.html>.
 - [2] MITRE. CWE-197: Numeric Truncation Error. <https://cwe.mitre.org/data/definitions/197.html>.
 - [3] MITRE. CWE-400: Uncontrolled Resource Consumption. <https://cwe.mitre.org/data/definitions/400.html>.
 - [4] MITRE. CWE-440: Expected Behavior Violation. <https://cwe.mitre.org/data/definitions/440.html>.
 - [5] MITRE. CWE-684: Protection Mechanism Failure. <https://cwe.mitre.org/data/definitions/693.html>.
 - [6] MITRE. CWE CATEGORY: 7PK - Security Features. <https://cwe.mitre.org/data/definitions/254.html>.
 - [7] MITRE. CWE CATEGORY: Behavioral Problems. <https://cwe.mitre.org/data/definitions/438.html>.
 - [8] MITRE. CWE CATEGORY: Numeric Errors. <https://cwe.mitre.org/data/definitions/189.html>.
 - [9] MITRE. CWE CATEGORY: Resource Management Errors. <https://cwe.mitre.org/data/definitions/399.html>.
 - [10] OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology
-